

Julia 进阶话题

Syslab 语言组

苏州同元软控信息技术有限公司

2024-10-12



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

课程须知

本课程课程目标：

本课程面向有一定 Julia 语言基础，想理解 Julia 的预编译机制、作用域；想了解在 Syslab 中 Julia 如何与 C/C++、Python 语言互调用、Syslab 静态代码生成、并行计算以及如何在 Syslab 中进行代码诊断和调试的用户。

本课程基于云化版本：Syslab 2024b 构建

本课程课程学习前，需要学习以下前置课程：

- Julia 基础语法
- Syslab 基本功能

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

目录

1. Julia 预编译

2. Julia 作用域

3. 外部语言接口

4. 静态代码生成

5. Julia 并行计算

6. 代码分析与调试

Syslab Julia 预编译

Syslab 语言组

苏州同元软控信息技术有限公司

2024-10-12



TONGYUAN
Software and Control

目录

1. 代码编译与执行机制

2. 预编译与编译延迟

01

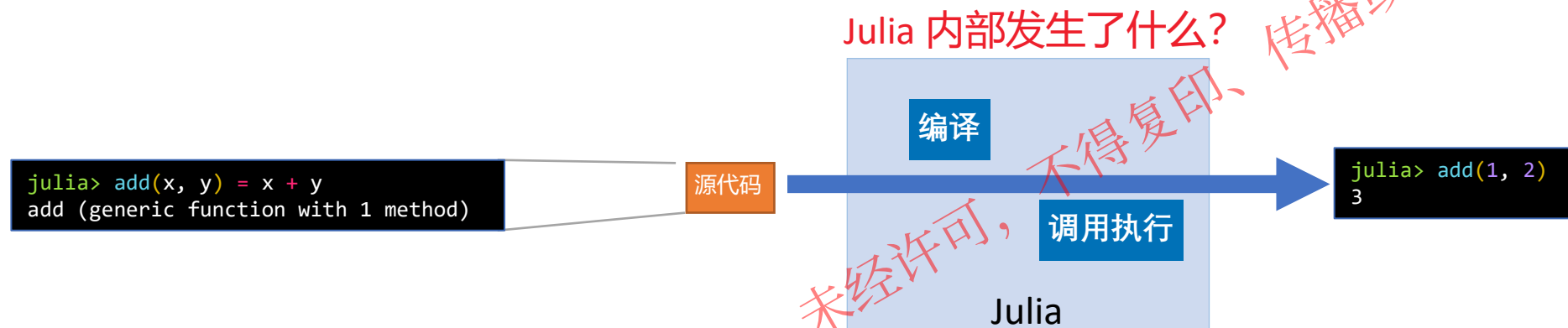
代码编译与执行机制

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

1. 代码编译与执行机制

Julia 是一门**动态编译型**语言：代码的编译与执行在同一个进程内完成



允许动态更新函数实现，验证数值结果

```
julia> add(x, y) = x + y
add (generic function with 1 method)

julia> add(1, 2)
3

julia> add(x, y) = x + y + 3
add (generic function with 1 method)

julia> add(1, 2)
6
```

Julia 编译器是一个运行时依赖：

- 代码的编译与执行均发生在用户侧
- 代码执行前进行按需代码编译
- 函数定义可以动态更新
(代码的编译结果会被自动更新)

1. 代码编译与执行机制

Julia 是一门**动态编译型**语言：函数的编译在第一次被调用时通过 LLVM 进行

```
julia> add(x, y) = x + y
add (generic function with 1 method)
```

```
julia> @code_llvm add(1, 2)
; @ REPL[1]:1 within `add`
define i64 @julia_add_236(i64 signext %0, i64 signext %1) #0 {
top:
; @ int.jl:87 within `+`
%2 = add i64 %1, %0
;
ret i64 %2
}
```

源代码

类型推断

Typed Code
(Julia IR)

@code_typed

LLVM 生成

LLVM IR

@code_llvm

LLVM 编译

可执行代码

@code_native

调用执行

```
julia> add(1, 2)
3
```

```
julia> @code_typed add(1, 2)
CodeInfo(
  %1 = Base.add_int(x, y)::Int64
  return %1
) => Int64
```

代码编译需要完整的类型信息才能发生，类型推断用于补充类型信息

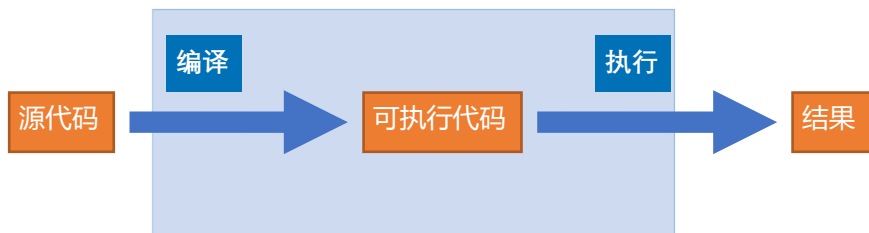
```
julia> @code_native add(1, 2)
.section      __TEXT,__text,regular,pure_instructions
.build_version macos, 13, 0
.globl _julia_add_409
.p2align     2
-- Begin function julia_add_409
_julia_add_409:
; @ REPL[2]:1 within `add`
; @ int.jl:87 within `+`
.cfi_startproc
; %top
; %bb.0:
add     x0, x1, x0
ret
.cfi_endproc
-- End function
.subsections_via_symbols
```

汇编代码

1. 代码编译与执行机制

Julia 函数调用遇到**新的类型组合**时会触发代码编译

函数编译的结果称为**方法实例**



Julia

代码编译在何时被触发?

```
julia> add(x, y) = x + y ✗
add (generic function with 1 method)

julia> add(1, 2) ✓
3

julia> add(3, 4) ✗
7

julia> add(1.0, 2.0) ✓
3.0

julia> add(1.0, 2) ✓
3.0
```

```
julia> using MethodAnalysis

julia> add(x, y) = x + y ✗
add (generic function with 1 method)

julia> methodinstances(add)
Core.MethodInstance[]

julia> add(1, 2) ✓
3

julia> methodinstances(add)
1-element Vector{Core.MethodInstance}:
MethodInstance for add(::Int64, ::Int64)

julia> add(3, 4) ✗
7

julia> methodinstances(add)
1-element Vector{Core.MethodInstance}:
MethodInstance for add(::Int64, ::Int64)

julia> add(1.0, 2.0) ✓
3.0

julia> methodinstances(add)
2-element Vector{Core.MethodInstance}:
MethodInstance for add(::Int64, ::Int64)
MethodInstance for add(::Float64, ::Float64)

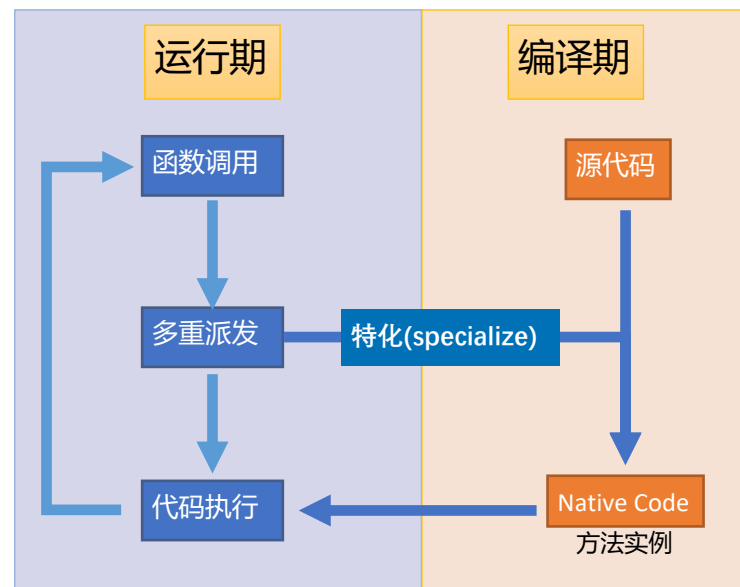
julia> add(1.0, 2) ✓
3.0

julia> methodinstances(add)
3-element Vector{Core.MethodInstance}:
MethodInstance for add(::Int64, ::Int64)
MethodInstance for add(::Float64, ::Float64)
MethodInstance for add(::Float64, ::Int64)
```

方法实例 (method instance)

- 方法实例具有完整的类型信息
- 编译新的方法实例的过程叫做**特化** (specialize)
- 如果方法实例已经存在, 则不需要重新编译
- 根据**全部**输入的类型来判断调用哪个方法实例的策略称为**多重派发** (multiple dispatch)

Julia 的函数调用、编译与执行围绕多重派发进行



✓ 触发编译 ✗ 未触发编译

1. 代码编译与执行机制

Julia **函数** (function) 由多个**方法** (method) 共同定义，多重派发调用最匹配的方法定义

不同参数（数目、类型）的组合对应的是同一个**函数**的不同**方法**

```
julia> is_number(x) = false
is_number (generic function with 1 method) 1

julia> is_number(x::Number) = true
is_number (generic function with 2 methods) 2

julia> is_number(x::Int) = true
is_number (generic function with 3 methods) 3

julia> is_number(1)
julia> is_number(1.0)
julia> is_number([2])
julia> is_number("hello world")
```

编译**方法实例**时会使用对应的方法

```
julia> methodinstances(is_number)
4-element Vector{Core.MethodInstance}:
MethodInstance for is_number(::Int64) 3
MethodInstance for is_number(::Float64) 2
MethodInstance for is_number(::Vector{Int64}) 1
MethodInstance for is_number(::String) 1
```

方法覆盖只发生在类型组合相同的情况下

```
julia> is_number(x::Number) = false 2
is_number (generic function with 3 methods)

julia> is_number(1) 3
true

julia> is_number(1.0) 2
false
```

1. 代码编译与执行机制

编译方法实例时会**递归**生成所需的全部方法实例

第一次调用时会编译哪些方法？是否会编译 `larger_than_6(::Int64)`？

```
julia> function count_number(A::AbstractArray)
    rst = 0
    for x in A
        if smaller_than_5(x)
            rst += 1
        elseif larger_than_6(x)
            rst += 1
        end
    end
    return rst
end
```

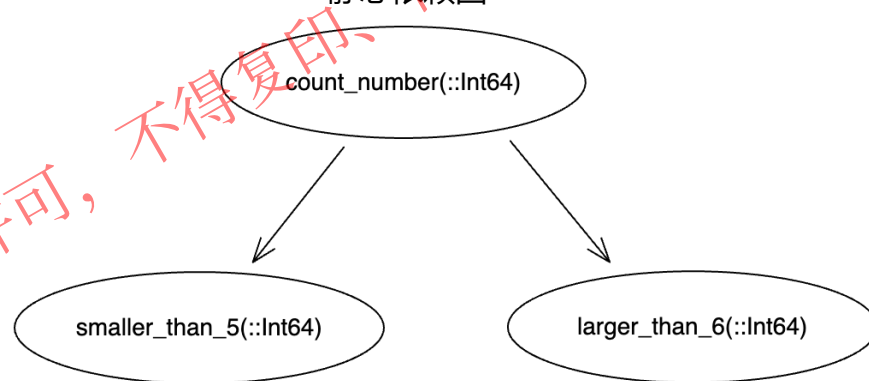
count_number (generic function with 1 method)

```
julia> smaller_than_5(x) = x < 5
smaller_than_5 (generic function with 1 method)
```

```
julia> larger_than_6(x) = x > 6
larger_than_6 (generic function with 1 method)
```

```
julia> count_number([1, 2, 3])
3
```

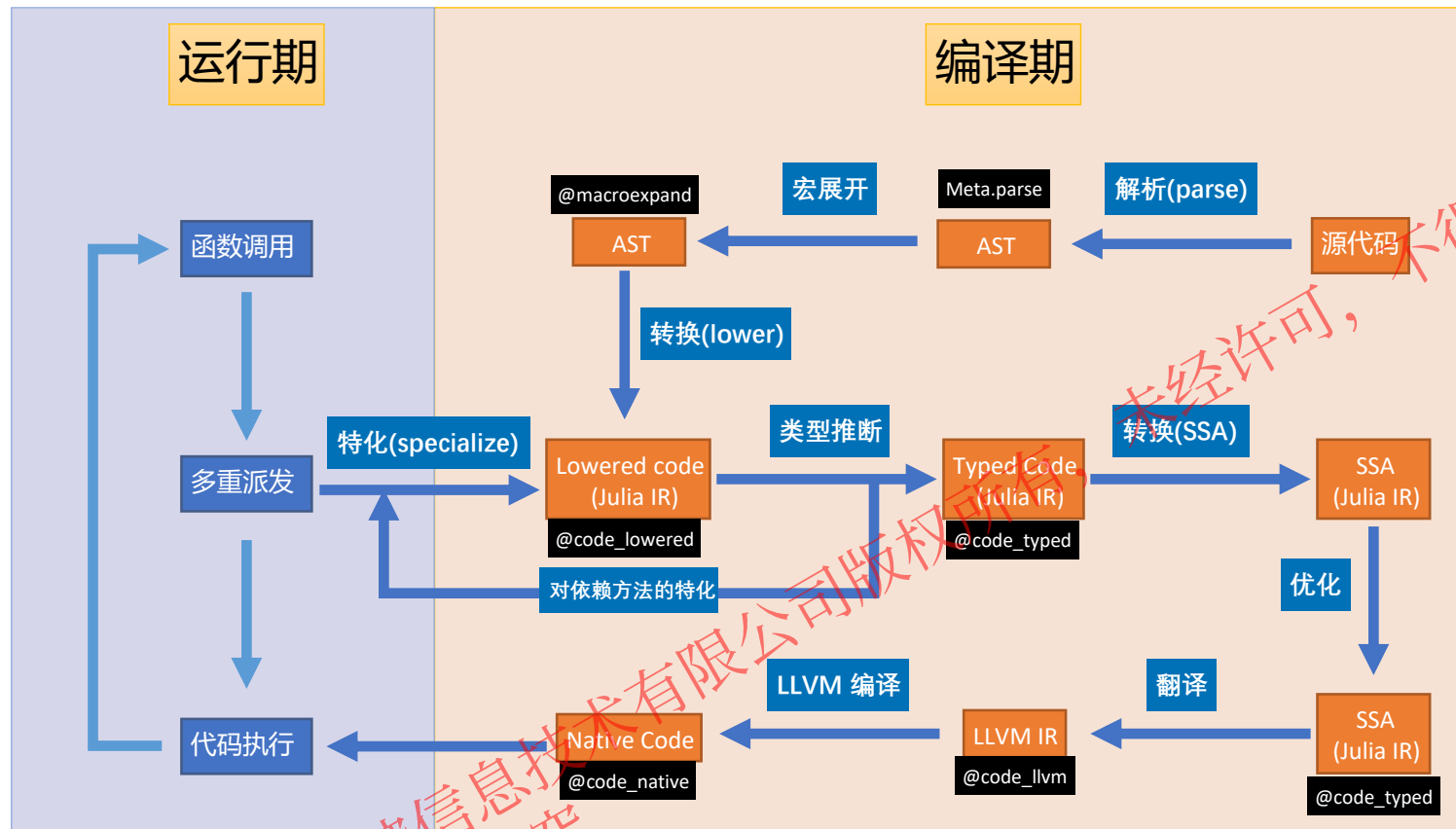
静态依赖图



```
julia> methodinstances(count_number)
1-element Vector{Core.MethodInstance}:
MethodInstance for count_number(::Vector{Int64})
julia> methodinstances(smaller_than_5)
1-element Vector{Core.MethodInstance}:
MethodInstance for smaller_than_5(::Int64)
julia> methodinstances(larger_than_6)
1-element Vector{Core.MethodInstance}:
MethodInstance for larger_than_6(::Int64)
```

1. 代码编译与执行机制 – 小结

Julia 代码编译与执行流程



Julia 是一门动态编译型语言

- 代码的编译与执行均发生在用户侧
- 代码执行前进行按需(递归)代码编译
- 编译的结果称为**方法实例 (method instance)**
- Julia **函数(function)** 由多个**方法(method)**组成
- Julia 通过**多重派发(multiple dispatch)** 确定具体的方法调用

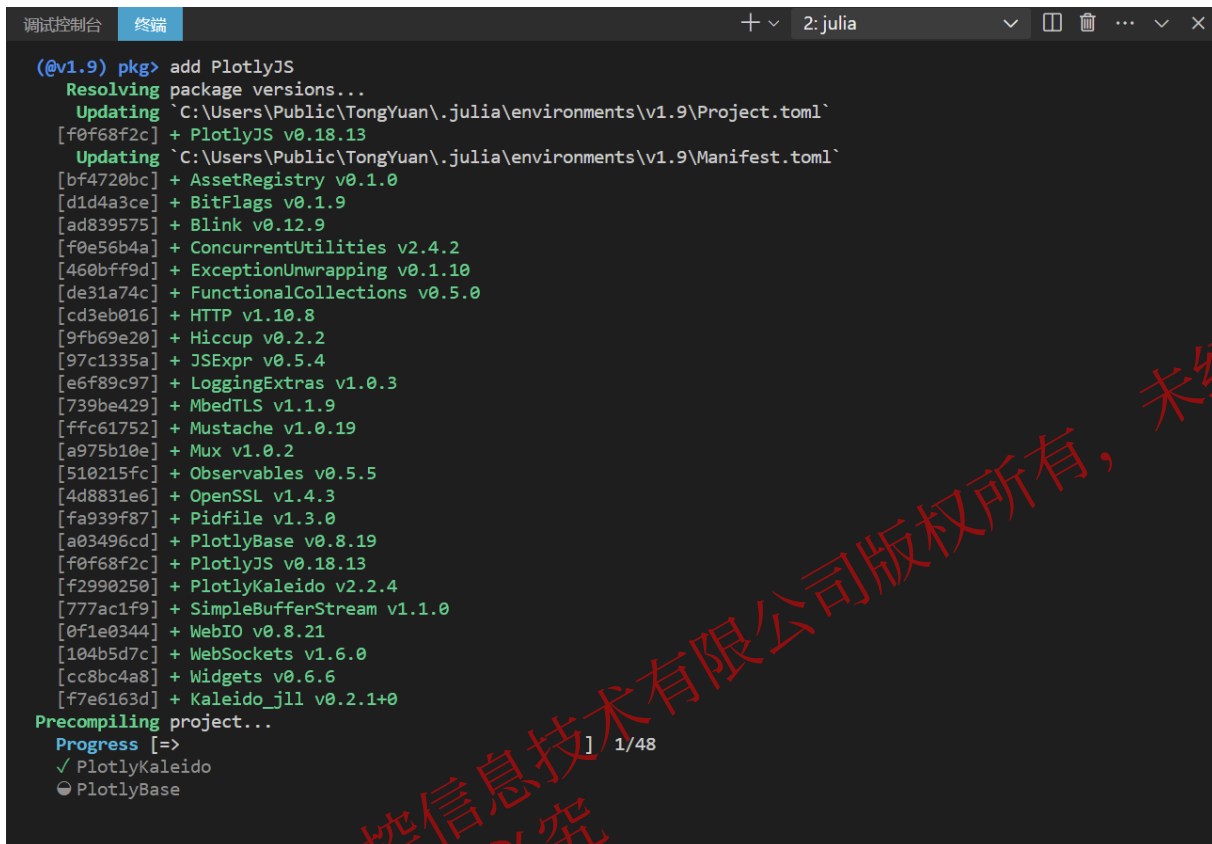
02 预编译与编译延迟

未经授权，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

2. 预编译与编译延迟 – 预编译

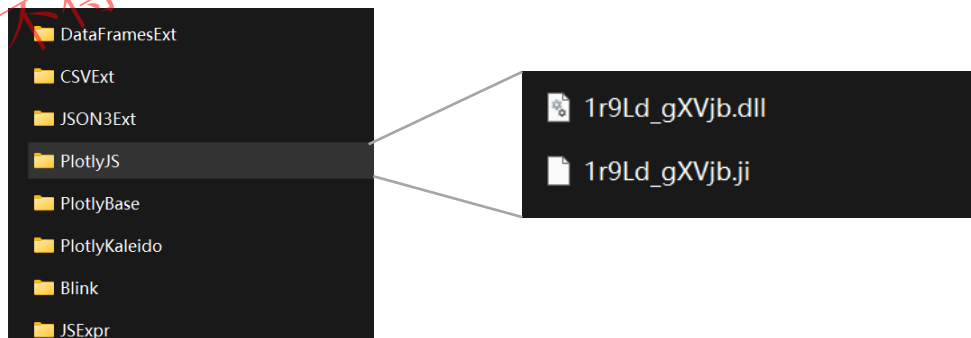
Julia 安装包时会**自动预编译 (precompile)**，该环节存储了一部分编译的结果。当某个包的任何一个依赖发生变更时，该包会在 using 或 import 时自动重新编译。



```
(@v1.9) pkg> add PlotlyJS
Resolving package versions...
Updating `C:\Users\Public\TongYuan\.julia\environments\v1.9\Project.toml`
[f0f68f2c] + PlotlyJS v0.18.13
Updating `C:\Users\Public\TongYuan\.julia\environments\v1.9\Manifest.toml`
[bf4720bc] + AssetRegistry v0.1.0
[d1d4a3ce] + BitFlags v0.1.9
[ad839575] + Blink v0.12.9
[f0e56b4a] + ConcurrentUtilities v2.4.2
[460bff9d] + ExceptionUnwrapping v0.1.10
[de31a74c] + FunctionalCollections v0.5.0
[cd3eb016] + HTTP v1.10.8
[9fb69e20] + Hiccup v0.2.2
[97c1335a] + JSEExpr v0.5.4
[e6f89c97] + LoggingExtras v1.0.3
[739be429] + MbedTLS v1.1.9
[ffc61752] + Mustache v1.0.19
[a975b10e] + Mux v1.0.2
[510215fc] + Observables v0.5.5
[4d8831e6] + OpenSSL v1.4.3
[fa939f87] + Pidfile v1.3.0
[a03496cd] + PlotlyBase v0.8.19
[f0f68f2c] + PlotlyJS v0.18.13
[f2990250] + PlotlyKaleido v2.2.4
[777ac1f9] + SimpleBufferStream v1.1.0
[0f1e0344] + WebIO v0.8.21
[104b5d7c] + WebSockets v1.6.0
[cc8bc4a8] + Widgets v0.6.6
[f7e6163d] + Kaleido_jll v0.2.1+0
Precompiling project...
Progress [=] 1/48
✓ PlotlyKaleido
● PlotlyBase
```

Julia 1.9 之前预编译仅储存了类型推断的结果；Julia 1.9 之后预编译还额外存储了 native code

预编译缓存：位于<.julia>/compiled/<v1.9>/包名



2. 预编译与编译延迟 – 预编译

预编译 (precompile) 需要提前确定要编译的方法信息

Julia 如何提前编译所需的方法？以及编译哪些方法？

```
module MyPkg

fast_mode() = get(ENV, "FAST_MODE")

end
```

```
julia> methodinstances(MyPkg.fast_mode)
Core.MethodInstance[]
```

```
module MyPkg

f(x) = x + 2

end
```

```
julia> methodinstances(MyPkg.f)
Core.MethodInstance[]
```

答：需要入口信息，Julia 才能开始推断

```
module MyPkg

fast_mode() = get(ENV, "FAST_MODE")

# 顶层模块执行触发 fast_mode 的预编译
if fast_mode()
    # do something
else
    # do something else
end

end
```

```
julia> methodinstances(MyPkg.fast_mode)
1-element Vector{Core.MethodInstance}:
MethodInstance for Main.MyPkg.fast_mode()
```

```
module MyPkg

f(x) = x + 2

# 主动告诉 Julia 预编译
# f(::Int) 与 f(::Float64) 方法
precompile(f, (Int, ))
precompile(f, (Float64, ))

end
```

```
julia> methodinstances(MyPkg.f)
2-element Vector{Core.MethodInstance}:
MethodInstance for Main.MyPkg.f(::Int64)
MethodInstance for Main.MyPkg.f(::Float64)
```


2. 预编译与编译延迟 – 编译失效

编译失效 (method invalidation) 会导致代码需要被重编译

第一次调用 `count_number` 会触发编译

```
julia> function count_number(A::AbstractArray)
    rst = 0
    for x in A
        if is_positive(x)
            rst += 1
        end
    end
    return rst
end

count_number (generic function with 1 method)

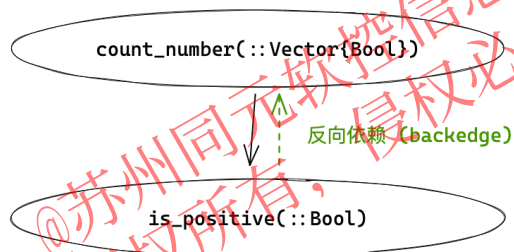
julia> is_positive(x) = x > 0
is_positive (generic function with 1 method)

julia> count_number(Bool[1, 0, 1])
2

julia> methodinstances(count_number)
1-element Vector{Core.MethodInstance}:
 MethodInstance for count_number(::Vector{Bool})

julia> methodinstances(is_positive)
1-element Vector{Core.MethodInstance}:
 MethodInstance for is_positive(::Bool)
```

静态依赖图



定义一个更具体的方法后, 旧的 `is_positive(::Bool)` 就失效了

```
julia> is_positive(x::Bool) = x
is_positive (generic function with 2 methods)

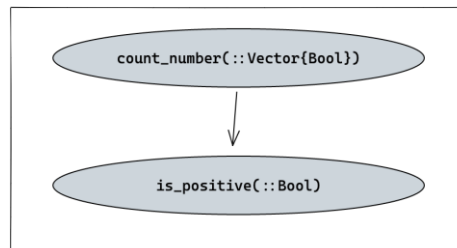
julia> count_number(Bool[1, 0, 1])
2

julia> methodinstances(is_positive)
2-element Vector{Core.MethodInstance}:
 MethodInstance for is_positive(::Bool)
 MethodInstance for is_positive(::Bool)

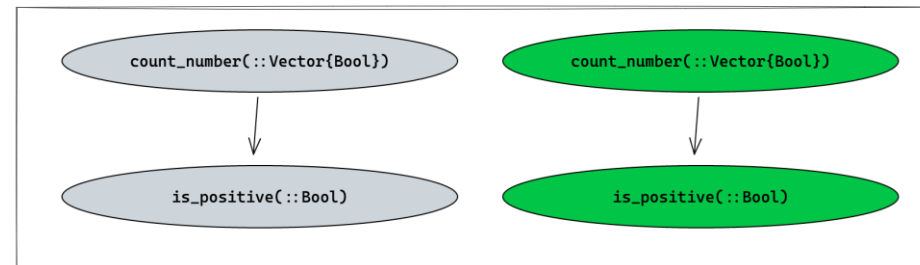
julia> methodinstances(count_number)
1-element Vector{Core.MethodInstance}:
 MethodInstance for count_number(::Vector{Bool})
```

更进一步地, 连带 `count_number(::Bool)` 也跟着失效了

时刻 1



时刻 2



旧的编译结果依然保留, 只是失效了

编译失效 (method invalidation)

- 编译失效源自于 Julia 允许动态增加与编译新的函数这一核心特性
- 由编译失效带来的重编译是导致 Julia 编译延迟的重要因素

```
julia> mi=methodinstances(is_positive)
2-element Vector{Core.MethodInstance}:
 MethodInstance for is_positive(::Bool)
 MethodInstance for is_positive(::Bool)

julia> mi[1].def
is_positive(x::Bool)
@ Main REPL[12]:1

julia> mi[2].def
is_positive(x)
@ Main REPL[5]:1
```


2. 预编译与编译延迟 – 类型海盗

类型海盗 (type piracy) 会很容易导致大面积编译失效，应该**禁止**该行为

如果不满足以下任意一个条件，则在 PkgA 中定义方法被称为**类型海盗 (type piracy)**:

- PkgA 拥有该函数
- PkgA 拥有输入参数中的任意一个类型

```
module MyPkg
  Base.log2(x::Int) = log(x)
end
```

MyPkg 并不拥有 Int 类型
✗ MyPkg 并不拥有 log2 函数

```
module MyPkg
  struct MyInt
    x::Int
  end
  Base.log2(x::MyInt) = log(x.x)
end
```

✓ MyPkg 拥有 MyInt 类型

```
module MyPkg
  ty_log2(x::Int) = log(x)
end
```

✓ MyPkg 拥有 ty_log2 函数

```
module MyPkg
  using LinearAlgebra
  LinearAlgebra.dot(x, y) = ...
end
```

MyPkg 并不拥有 Any 类型
✗ MyPkg 并不拥有 dot 函数

在 MyPkg 中修改 LinearAlgebra 的 dot 函数定义

```
module MyPkg
  ty_dot(x, y) = ...
end
```

✓ MyPkg 拥有 ty_dot 函数

```
module MyPkg
  dot(x, y) = ...
end
```

在 MyPkgMLang 中定义一个重名函数 dot

✓ MyPkgMLang 拥有 dot 函数

2. 预编译与编译延迟 – 类型稳定

类型稳定 (type stable): 函数的返回值及中间结果的类型可以根据输入的类型（而非值）来推断。

✗ f 可能返回 Int 也可能返回 Float64

```
julia> function f(x::Float64)
    if x < 0.5
        return Int(0)
    else
        return Float64(1.0)
    end
end
f (generic function with 1 method)
```

```
julia> @code_warntype f(1.0)
MethodInstance for f(::Float64)
  from f(x::Float64) @ Main REPL
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Union{Float64, Int64}
  1 -- %1 = (x < 0.5)::Bool
  └─ goto #3 if not %1
```

✓ 返回值的类型一定与 x 的类型相同

```
julia> function f(x)
    if x < 0.5
        return zero(x)
    else
        return one(x)
    end
end
f (generic function with 1 method)
```

```
julia> @code_warntype f(1.0)
MethodInstance for f(::Float64)
  from f(x::Float64) @ Main REPL
Arguments
  #self#::Core.Const(f)
  x::Float64
Locals
  y::Float64
Body::Float64
```

✗ f 的中间结果 y 可能为 Int 也可能为 Float64

```
julia> function f(x::Float64)
    if x < 0.5
        y = 1
    else
        y = x^2
    end
    return Float64(y)
end
f (generic function with 1 method)
```

```
julia> @code_warntype f(1.0)
MethodInstance for f(::Float64)
  from f(x::Float64) @ Main REPL
Arguments
  #self#::Core.Const(f)
  x::Float64
Locals
  y::Union{Float64, Int64}
Body::Float64
  1 -- Core.NewvarNode(: (y))
  %2 = (x < 0.5)::Bool
  └─ goto #3 if not %2
```

✓ f 的中间结果 y 都为 Float64 类型

```
julia> function f(x::Float64)
    if x < 0.5
        y = 1.0
    else
        y = x^2
    end
    return Float64(y)
end
f (generic function with 1 method)
```

```
julia> @code_warntype f(1.0)
MethodInstance for f(::Float64)
  from f(x::Float64) @ Main REPL
Arguments
  #self#::Core.Const(f)
  x::Float64
Locals
  y::Float64
Body::Float64
```

✗ rst 的类型可能为 Int 也可能为 Float64
取决于 A 是否为空矩阵

```
julia> function mysum(A::Array{Float64})
    rst = 0
    for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)
```

```
julia> @code_warntype mysum([1.5,2])
MethodInstance for mysum(::Vector{Float64})
  from mysum(A::Array{Float64}) @ Main REPL
Arguments
  #self#::Core.Const(mysum)
  A::Vector{Float64}
Locals
  @_3::Union{Nothing, Tuple{Int64, Int64}}
  rst::Union{Float64, Int64}
  i::Int64
Body::Union{Float64, Int64}
```

✓ rst 的类型一定为 A 的元素类型

```
julia> function mysum(A::AbstractArray)
    rst = zero(eltype(A))
    for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)
```

@_3 是一个迭代器状态变量，并不会影响你的函数的类型稳定性

```
julia> @code_warntype mysum([1.5,2.5])
MethodInstance for mysum(::Vector{Float64})
  from mysum(A::AbstractArray{Float64}) @ Main
Arguments
  #self#::Core.Const(mysum)
  A::Vector{Float64}
Locals
  @_3::Union{Nothing, Tuple{Int64, Int64}}
  rst::Float64
  i::Int64
Body::Float64
```

2. 预编译与编译延迟 – 类型稳定

类型不稳定的代码无法提前编译出所有可能方法，甚至可能会比动态语言还要慢。

执行 `f(1, 2)` 之前无法判断需要编译哪些方法实例

```
julia> function get_number(x, nbits)
    if nbits == 16
        Float16(x)
    elseif nbits == 32
        Float32(x)
    elseif nbits == 64
        Float64(x)
    elseif nbits > 64
        BigFloat(x)
    end
end
get_number (generic function with 1 method)

julia> add(x, y) = x + y
add (generic function with 1 method)

julia> function f(x, y)
    nbits = rand((16 32 64 128))
    x = get_number(x, nbits)
    y = get_number(y, nbits)
    return add(x, y)
end
f (generic function with 1 method)
```

```
julia> methodinstances(add)
Core.MethodInstance[]

julia> f(1, 2)
3.0

julia> methodinstances(add)
2-element Vector{Core.MethodInstance}:
 MethodInstance for add(::Any, ::Any)
 MethodInstance for add(::BigFloat, ::BigFloat)

julia> f(1, 2)
3.0

julia> methodinstances(add)
2-element Vector{Core.MethodInstance}:
 MethodInstance for add(::Any, ::Any)
 MethodInstance for add(::BigFloat, ::BigFloat)

julia> f(1, 2)
3.0

julia> methodinstances(add)
3-element Vector{Core.MethodInstance}:
 MethodInstance for add(::Any, ::Any)
 MethodInstance for add(::BigFloat, ::BigFloat)
 MethodInstance for add(::Float64, ::Float64)
```

类型不稳定的代码对预编译不友好:

- 类型不稳定的代码会导致额外的编译工作量
- 类型不稳定的代码的执行会导致新方法的动态编译

2. 预编译与编译延迟 – 类型稳定

鼓励**类型稳定**的Julia代码，可以显著提升运行时性能。

类型不稳定的代码

```
julia> using BenchmarkTools

julia> function mysum(A)
    rst = 0
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end

mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum($A);
1.869 ms (0 allocations: 0 bytes)
```

输入矩阵 A 是否为空矩阵会影响返回值的类型

```
julia> @code_warntype mysum(rand(4, 4))
MethodInstance for mysum(::Matrix{Float64})
  from mysum(A; dims) in Main at REPL[27]:1
Arguments
  #self#::Core.Const(mysum)
  A::Matrix{Float64}
Locals
  @_3::Union{Nothing, Tuple{Int64, Int64}}
  rst::Union{Float64, Int64}
  i::Int64
Body::Union{Float64, Int64}
1 --      (rst = 0)
   %2  = Main.eachindex(A)::Base.OneTo{Int64}
   (@_3 = Base.iterate(%2))
   %4  = (@_3 === nothing)::Bool
   %5  = Base.not_int(%4)::Bool
        goto #4 if not %5
2 --- %7  = @_3::Tuple{Int64, Int64}
        (i = Core.getfield(%7, 1))
   %9  = Core.getfield(%7, 2)::Int64
   %10 = rst::Union{Float64, Int64}
   %11 = Base.getindex(A, i)::Float64
        (rst = %10 + %11)
        (@_3 = Base.iterate(%2, %9))
   %14 = (@_3 === nothing)::Bool
   %15 = Base.not_int(%14)::Bool
        goto #4 if not %15
3 --      goto #2
4 ---      return rst
```

类型稳定的代码

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end

mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum($A);
131.004μs (0 allocations: 0 bytes)
```

2. 预编译与编译延迟 – 小结

预编译可以将重要函数的编译工作提前

- Julia装包时的预编译环节存储了一部分编译的结果
- 预编译需要程序员提供编译入口信息 -- Julia函数就像C++ 模板

鼓励**类型稳定**的Julia代码

- 类型不稳定与抽象类型会影响运行时性能
- 类型不稳定与抽象类型对于预编译不友好 -- 动态派发
- **动态派发**是指在代码运行时（而非编译期间）根据实际数值的类型动态地决定要调用的方法

Julia **编译失效**导致的重编译是延迟的一个主要来源

- Julia1.9的加载时间很难优化到静态语言的程序：
因为在加载时需要额外处理编译失效现象
- 编译失效源自于Julia允许动态增加与编译新的函数这一核心特性
- **类型海盗**容易导致大面积编译失效问题，尽量避免该行为

Syslab Julia 作用域

平文婷

苏州同元软控信息技术有限公司

2024-10-15



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. 作用域概念和意义

2. Julia 的变量作用域

3. 避免作用域引起的问题

01

作用域概念和意义

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

侵权必究

1.作用域概念和意义

- 变量的 **作用域** 是某个变量可见的范围，变量作用域使得同名的不同变量能够区分开来。
- 相同的变量名是否指向同一个东西的规则被称为作用域规则。
- 给变量划分作用域有助于**解决变量命名冲突**。

语言	作用域类型
Matlab	全局，局部作用域
Python	L(local), G(global), E(enclosing), B(built-in)
C	代码块作用域，文件作用域，函数原型作用域，函数作用域
C++	全局，函数，块作用域，类作用域...

1.作用域概念和意义

解决 Julia 使用过程中由于
作用域导致的问题

帮助用户从 Matlab 顺利的切换到 Syslab

深入理解 Julia 的作用域，了解问题的原因

```
test.jl
1 z = 0
2 for i=1:5
3     z = i
4 end
5 println(z)
6

julia> 正在运行 test.jl
Warning: Assignment to `z` in soft scope is ambiguous because a global variable by the same name exists: `z` will be treated as a new local. Disambiguate by using `local z` to suppress this warning or `global z` to assign to the existing global variable.
@ c:\Users\TR\Desktop\Penpen\M\Scope Variables\test.jl:3
0

julia> z = 0
0

julia> for i=1:5
        z = i
    end

julia> println(z)
5

julia>
```

Julia env: v1.9 行 6, 列 1 空格: 4 UTF-8 CRLF Julia Main

- 为什么会在脚本中运行会产生该 warning 信息?
- 为什么在脚本和命令行中运行结果不同?
- 如何避免这样的问题?

02 Julia 的变量作用域

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

2. Julia的变量作用域

在 Julia 中主要有两种作用域，全局作用域 与 局部作用域，可用关键字 `global`, `local` 声明。

结构	作用域类型	允许使用在
<code>module</code> , <code>baremodule</code> , <code>REPL</code>	全局	全局
<code>struct</code>	局部 (软)	全局
<code>for</code> , <code>while</code> , <code>try</code>	局部 (软)	全局, 局部
<code>macro</code>	局部 (硬)	全局
函数, <code>do</code> 语句块, <code>let</code> 语句块	局部 (硬)	全局, 局部

注意：同一个作用域内，每个变量的作用必须是唯一的，不能说在第一个语句是读取父作用域的同名变量值，第二个语句就因为赋值变成了该作用域的局部变量。

2. Julia的变量作用域 – 全局作用域

- 每一个模块有自己的全局作用域，但是没有有一个在所有模块中都起作用的统一的全局作用域。模块内在所有函数定义外部赋值的变量为全局变量。模块内的任何位置用 `global` 关键字声明的变量为该模块的全局变量。
- 在命令行环境中定义的变量属于命令行环境的全局作用域，实际是 Main 模块的全局作用域。
- 为了访问其它模块的全局变量，可以用 `using` 或者 `import` 引入，也可以用“模块名.变量名”格式。

示例：每个模块有自己全局作用域，访问可以采用import引用

```
module Amod
    x = 1
    foo() = x
end
import .Amod
x = -1
println(Amod.foo())
## 1
```

最后一个语句调用 `Amod.foo()` 时用到的是模块 `Amod` 的全局变量 `x`，这是定义 `foo()` 时包含函数定义的代码中的 `x` 变量。

赋值为-1的变量 `x` 是命令行的全局变量，这是调用 `foo()` 时所处环境中的变量。

`using` 和 `import` 的详细介绍可以查看《Julia 语法入门》课程的第四小节——模块化

2. Julia的变量作用域 – 局部作用域

许多代码块结构都会产生一个局部作用域，如函数定义、for 循环等。局部作用域都可以读写其定义环境所处作用域（称为父作用域）的变量，但有例外：

- 如果在局部作用域中用 local 声明了变量，则对该变量的修改不会影响父作用域的变量。
- 局部作用域不是命名空间，所以内层可以访问外层变量，但是外层无论如何不能访问内层作用域的局部变量。
- 如果希望在局部作用域修改**全局**变量：
 - 非交互式环境：需要在局部作用域中用 global 声明该变量；
 - 可交互式环境：硬作用域需要用 global 声明该变量，软作用域不需要。

根据是否允许使用相同的变量名覆盖全局变量，局部作用域分成硬局部作用域和软局部作用域。

2. Julia的变量作用域 – 局部作用域

局部作用域：
局部变量无法修改全局变量

问题：在交互式环境中赋值操作不方便

在交互式环境中：
是否允许使用相同的变量名覆盖全局变量



硬作用域：硬作用域中的变量无法覆盖全局变量

软作用域：在交互式环境中允许使用相同的变量名覆盖全局变量

```
C:\Users\TR\AppData\Local\Programs\Julia\Julia-1.4.2\bin\julia.exe  命令提示符 - julia
Documentation: https://docs.julialang.org
Type "?" for help, "]?" for Pkg help.
Version 1.4.2 (2020-05-23)
Official https://julialang.org/ release

julia> z = 0;
julia> for i=1:5
           z = i
       end
julia> println(z)
0
julia>
```

```
C:\Users\TR\AppData\Local\Programs\Julia\Julia-1.9.3\bin\julia.exe  命令提示符 - julia
Documentation: https://docs.julialang.org
Type "?" for help, "]?" for Pkg help.
Version 1.9.3 (2023-08-24)
Official https://julialang.org/ release

julia> z = 0;
julia> for i=1:5
           z = i
       end
julia> println(z)
5
julia>
```

2. Julia的变量作用域 – 局部作用域：硬作用域

硬作用域里的变量不会对全局作用域里的变量产生影响

- function

```
function foo()  
    x=2  
    return x  
end  
x=1  
foo()  
@show x
```

```
调试控制台 终端 + Julia REPL (v1.9.3) [ ] [X] ... ^ X  
  
julia> function foo()  
           x=2  
           return x  
       end  
foo (generic function with 1 method)  
  
julia> x=1  
1  
  
julia> foo()  
2  
  
julia> @show x  
x = 1  
1
```

- let结构

```
z = -1  
let x = 1, y = 2  
    z = x + y  
    @show z  
end  
@show z
```

```
调试控制台 终端 + Julia REPL (v1.9.3) [ ] [X] ... ^ X  
  
julia> z = -1  
-1  
  
julia> let x = 1, y = 2  
           z = x + y  
           @show z  
       end  
  
z = 3  
3  
  
julia> @show z  
z = -1  
-1
```


2. Julia的变量作用域 – 局部作用域：软作用域

软作用域里的变量可能对全局作用域里的变量产生影响：

- 如果全局变量 z 是**未定义**，最终此次赋值会在该作用域创建一个名为 z 的新局部变量，见示例1。
- 如果全局变量 z 是**已定义**：
 - 在**非交互**的上下文（文件、eval、include）中，会打印一个有歧义警告，同时创建一个新局部变量，见示例2；
 - 在**交互**的上下文（REPL, notebooks）中，会向全局变量 z 赋值，见示例3。

示例1

```
julia> @isdefined(z)
false

julia> for i = 1:5
           z = i
       end

julia> @isdefined(z)
false
```

示例2

```
z = 0
for i = 1:5
    z = i
end
println(z)
```

调试控制台 终端

Julia REPL (v1.9.3)

```
julia> 正在运行 test.jl
Warning: Assignment to `z` in soft scope is ambiguous because a global variable by the same name exists: `z` will be treated as a new local. Disambiguate by using `local z` to suppress this warning or `global z` to assign to the existing global variable.
@ c:\Users\TR\Desktop\Penpen\M\PreloadMPackages\TyPlot\2-D and 3-D Plots\Discrete Data Plots\stem\test.jl:3
0
```

示例3

```
julia> z = 0
0

julia> for i = 1:5
           z = i
       end

julia> println(z)
5
```

2. Julia的变量作用域 – 局部作用域嵌套

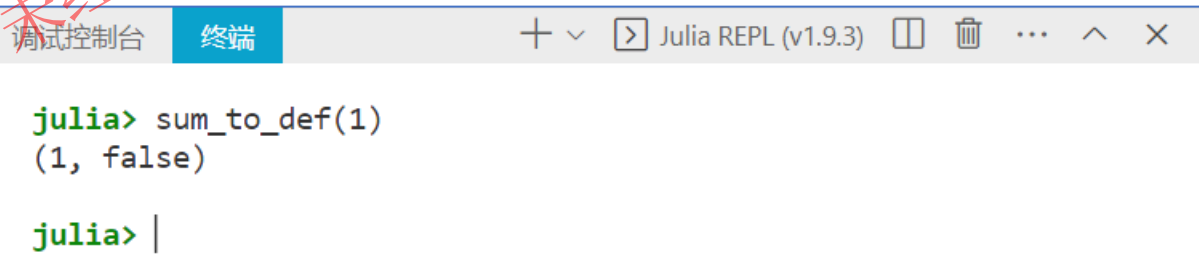
- 内层作用域寻找外层作用域的变量可以找到。
- 外层作用域寻找内层作用域的变量不可以找到。

示例：

for 循环语句中可以找到 function 中定义的变量 s

function 中无法找到 for 循环中定义的变量 t

```
function sum_to_def(n)
    s = 0 # new local
    for i = 1:n
        t = s + i # new local `t`
        s = t # assign existing local `s`
    end
    return s, @isdefined(t)
end
sum_to_def(1)
```



The screenshot shows a Julia REPL window with the title 'Julia REPL (v1.9.3)'. The terminal output shows the function call `sum_to_def(1)` returning `(1, false)`. The prompt `julia>` is visible at the bottom.

03

避免作用域引起的问题

@苏州同元软控信息技术有限公司版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

2.避免作用域引起的问题

1. 如何在局部作用域使用全局变量？

新建变量：

在局部作用域创建局部变量，将全局变量赋值给局部变量。

```
s = 0 # global `s`  
for i = 1:5  
    y = s + i # new local `y`  
    println(y)  
end
```

调试控制台

终端

+ v [x] Julia REPL (v1.9.3) [icon] [icon] ... ^ x

julia> 正在运行 test.jl

1
2
3
4
5

2.避免作用域引起的问题

2. 如何在局部作用域修改全局变量？

使用 local 和 global 标识符：

能够帮助 Julia 准确判断用户想使用的变量所在的作用域、提高代码的可读性。

```
s = 0 # global `s`  
for i = 1:5  
    s = i # new local `s`  
    println("Inside for loop: s=$s")  
end  
println("Outside for loop: s=$s")
```

```
s = 0 # global `s`  
for i = 1:5  
    global s  
    s = i # assign global `s`  
    println("Inside for loop: s=$s")  
end  
println("Outside for loop: s=$s")
```

```
调试控制台 终端 + Julia REPL (v1.9.3)  
julia> 正在运行 test.jl  
[ Warning: Assignment to `s` in soft scope is ambiguous because a glob  
al variable by the same name exists: `s` will be treated as a new loca  
l. Disambiguate by using `local s` to suppress this warning or `global  
s` to assign to the existing global variable.  
L @ c:\Users\TR\Desktop\Penpen\M\Scope Variables\test.jl:3  
Inside for loop: s=1  
Inside for loop: s=2  
Inside for loop: s=3  
Inside for loop: s=4  
Inside for loop: s=5  
Outside for loop: s=0
```

```
调试控制台 终端 + Julia REPL (v1.9.3)  
julia> 正在运行 test.jl  
Inside for loop: s=1  
Inside for loop: s=2  
Inside for loop: s=3  
Inside for loop: s=4  
Inside for loop: s=5  
Outside for loop: s=5
```

2.避免作用域引起的问题

3. 如何避免全局作用域的和软局部作用域里的变量相互影响？

封装成函数：

脚本文件封成函数能够隔绝全局作用域，避免全局作用域的和软局部作用域里的变量相互影响。

```
s = 0 # global `s`  
for i = 1:5  
    s = s + i # local `s`  
    println("Inside for loop: s=$s")  
end  
println("Outside for loop: s=$s")
```



```
function foo()  
    s = 0 # local `s`  
    for i = 1:5  
        s = s + i # assign existing local `s`  
        println("Inside for loop: s=$s")  
    end  
    println("Outside for loop: s=$s")  
end  
foo()
```

```
调试控制台 终端  
+ Julia REPL (v1.9.3)  
julia> 正在运行 test.jl  
[ Warning: Assignment to `s` in soft scope is ambiguous because a global variable by the same name exists: `s` will be treated as a new local. Disambiguate by using `local s` to suppress this warning or `global s` to assign to the existing global variable.  
@ c:\Users\TR\Desktop\Penpen\M\Scope Variables\test.jl:3  
ERROR: UndefVarError: `s` not defined  
Stacktrace:  
[1] top-level scope  
@ c:\Users\TR\Desktop\Penpen\M\Scope Variables\test.jl:3
```

```
调试控制台 终端  
+ Julia REPL (v1.9.3)  
julia> 正在运行 test.jl  
Inside for loop: s=1  
Inside for loop: s=3  
Inside for loop: s=6  
Inside for loop: s=10  
Inside for loop: s=15  
Outside for loop: s=15
```

Syslab 外部语言接口

Syslab 语言组

苏州同元软控信息技术有限公司

2024-10-17



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. Julia与C/C++互调用

2. Julia与Python互调用

01

Julia与C/C++互调用

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

1.1 Julia 调用 C/C++

在数值计算领域，存在很多用C或Fortran写的高质量且成熟的库，为了便捷复用现有资产，Julia提供简洁且高效的调用方式，不需要任何“胶水”代码。

可以使用 `ccall` 来生成一个对C/C++库函数的调用。`ccall`定义如下：

```
ccall(function_name, library), returntype, (argtype1, ...), argvalue1, ...)  
ccall(function_name, returntype, (argtype1, ...), argvalue1, ...)  
ccall(function_pointer, returntype, (argtype1, ...), argvalue1, ...)
```

注：`ccall`是Julia内置库Base的关键字，无需加载额外包

例如：

```
ccall(:clock, Int32, ())
```

调用C库里面的`clock`函数，空参，返回值类型为`Int32`

1.1 Julia 调用 C/C++

对于C标准库中的函数，Julia可以直接调用。

例如，C标准库中的getenv函数声明：char* getenv (const char* name);

示例：

```
ccall(function_name, returntype, (argtype1, ...), argvalue1, ...)
```

函数名 返回值类型 参数类型 实参

```
julia> path = ccall(:getenv, Cstring, (Cstring,), "PATH")#注：函数名前面加冒号表示symbol类型
```

输出

```
Cstring(0x000001cbf6e05142)
```

```
julia> unsafe_string(path)#Copy a string from the address of a C-style, 表示从C地址中取字符串值
```

输出

```
"/usr/local/share/TongYuan/julia-1.9.3/bin:/usr/local/share/TongYuan/julia-1.9.3/lib:/usr/local/share/TongYuan/julia-1.9.3/lib/julia:/usr/bin:/usr/local/share/TongYuan/.julia/miniforge3:/usr/local/share/TongYuan/.julia/miniforge3/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
```

1.1 Julia 调用 C/C++

还可以使用另外一种方法 @ccall 来调用C函数，更加简洁清晰。和上一种方法使用等价

@ccall 的定义如下：

```
@ccall library.function_name(argvalue1::argtype1, ...)::returntype  
@ccall function_name(argvalue1::argtype1, ...)::returntype  
@ccall $function_pointer(argvalue1::argtype1, ...)::returntype
```

例如：

函数名 实参 参数类型 返回值类型

↑ ↑ ↑ ↑

```
julia> path = @ccall getenv("PATH"::Cstring)::Cstring
```

```
julia> unsafe_string(path)
```

1.1 Julia 调用 C/C++

除了可以直接调用C标准库外，还可以调用用户开发的C++动态库。

假设，用户开发了一个动态库ArrayMaker.dll，该动态库的C/C++头文件定义如下：

```
#ifndef ARRAYMAKER_H
#define ARRAYMAKER_H

#include "ArrayMaker_global.h"

struct ArrayMaker {
    int nNumber;
    double* pArray;
};

// 求和
extern "C" ARRAYMAKER_EXPORT double GetSum(double x, double y);

extern "C" ARRAYMAKER_EXPORT ArrayMaker* CreateObj();
extern "C" ARRAYMAKER_EXPORT void DeleteObj(ArrayMaker ** pobj);
extern "C" ARRAYMAKER_EXPORT double* FillArray(ArrayMaker* pobj, int num, double value);
extern "C" ARRAYMAKER_EXPORT double* SetValue(ArrayMaker* pobj, int nth/*base-1*/, double value);
extern "C" ARRAYMAKER_EXPORT int GetValues(ArrayMaker * pobj, double* out, int len);

#endif
```

1.1 Julia 调用 C/C++

除了可以直接调用C标准库外，还可以调用用户开发的C++动态库。

通过 **Libdl** 来实现加载和卸载C/C++动态库。libdl函数库为Julia内置函数库，无需手动安装。

```
using Libdl

# 加载库
lib_path = joinpath(@__DIR__, "ArrayMaker", "libtest") # 用户动态库路径, 如 .../libtest.so
lib = Libdl.dlopen(lib_path)

# 获取调用函数的符号
GetSum = Libdl.dlsym(lib, :GetSum)

# 调用函数
c = @ccall $GetSum(2::Cdouble, 3::Cdouble)::Cdouble

...

# 关闭dll
Libdl.dlclose(lib)
```

函数指针

与调用C标准库方法一样

使用的函数帮助文档链接: <https://juliacn.gitlab.io/JuliaZH.jl/stdlib/Libdl.html>

1.1 Julia 调用 C/C++

目前，基本的C/C++值类型可以转换为Julia类型，以C为前缀。标准Julia别名与Julia基本类型没有区别

C 类型	Fortran 类型	标准 Julia 别名	Julia 基本类型
unsigned char	CHARACTER	Cuchar	UInt8
bool (Bool in C99+)		Cuchar	UInt8
short	INTEGER*2, LOGICAL*2	Cshort	Int16
unsigned short		Cushort	UInt16
int, BOOL (C, typical)	INTEGER*4, LOGICAL*4	Cint	Int32
unsigned int		Cuint	UInt32
long long	INTEGER*8, LOGICAL*8	Clonglong	Int64
unsigned long long		Culonglong	UInt64
intmax_t		Cintmax_t	Int64
uintmax_t		Cuintmax_t	UInt64
float	REAL*4i	Cfloat	Float32
double	REAL*8	Cdouble	Float64
complex float	COMPLEX*8	ComplexF32	Complex{Float32}
complex double	COMPLEX*16	ComplexF64	Complex{Float64}
ptrdiff_t		Cptrdiff_t	Int
ssize_t		Cssize_t	Int
size_t		Csize_t	UInt

C 类型	Fortran 类型	标准 Julia 别名	Julia 基本类型
void			Cvoid
void and [[noreturn]] or _Noreturn			Union{}
void*			Ptr{Cvoid} (或类似的 Ref{Cvoid})
T* (where T represents an appropriately defined type)			Ref{T} (只有当 T 是 isbits 类型时, T 才可以安全地转变)
char* (or char[] e.g. a string)	CHARACTER*N		Cstring if NUL-terminated, or Ptr{UInt8} if not
char** (or *char[])			Ptr{Ptr{UInt8}}
j1_value_t* (any Julia Type)			Any
j1_value_t* const* (一个 Julia 值的引用)			Ref{Any} (常量, 因为转变需要写屏障, 不可能正确插入)
va_arg			Not supported
... (variadic function specification)			T... (其中 T 是上述类型之一, 当使用 ccall 函数时)
... (variadic function specification)			; va_arg1::T, va_arg2::S 等 (仅支持@ccall 宏)

上面标注红框为常用类型

1.1 Julia 调用 C/C++

Julia调用C/C++的一个完整示例

① Julia代码:

```
using Libdl

# 加载dll
lib_path = joinpath(@__DIR__, "ArrayMaker", "libtest")
lib = Libdl.dlopen(lib_path)

# 获取符号
CreateObj = Libdl.dlsym(lib, :CreateObj)
DeleteObj = Libdl.dlsym(lib, :DeleteObj)
FillArray = Libdl.dlsym(lib, :FillArray)

# 创建对象指针
pobj = @ccall $CreateObj()::Ptr{Cvoid}

# 填充数组
len = 5
parr = @ccall $FillArray(pobj::Ptr{Cvoid}, len::Cint, 3.5::Cdouble)::Ptr{Cdouble}
arr = [unsafe_load(parr, i) for i = 1:len]
#=
5-element Vector{Float64}:
 3.5
 3.5
 3.5
 3.5
 3.5
=#

# 销毁对象
@ccall $DeleteObj(Ref{Ptr{Cvoid}}{pobj})::Cvoid
pobj = C_NULL

# 关闭dll
Libdl.dlclose(lib)
```

② C/C++头文件代码:

```
#ifndef ARRAYMAKER_H
#define ARRAYMAKER_H

#include "ArrayMaker_global.h"

struct ArrayMaker {
    int nNumber;
    double* pArray;
};

extern "C" ARRAYMAKER_EXPORT ArrayMaker* CreateObj();
extern "C" ARRAYMAKER_EXPORT void DeleteObj(ArrayMaker ** ppobj);
extern "C" ARRAYMAKER_EXPORT double* FillArray(ArrayMaker* pobj, int num,
double value);

#endif
```

备注:

CreateObj: 创建数组构造器对象;

DeleteObj: 销毁数组构造器对象;

FillArray: 填充数组, 输入参数为数组长度、数组元素值。初始时, 所有数据元素值都为value。

1.1 Julia 调用 C/C++

Julia调用C/C++的一个完整示例

③ C/C++源文件代码：

```
#include "ArrayMaker.h"
#include <math.h>

using namespace std;

ArrayMaker* CreateObj()
{
    auto p = new ArrayMaker;
    p->nNumber = 0;
    p->pArray = nullptr;
    return p;
}

void DeleteObj(ArrayMaker** ppobj)
{
    if (ppobj == nullptr) {
        return;
    }

    auto& pobj = *ppobj;
    if (pobj != nullptr) {

        //删除数据
        if (pobj->pArray != nullptr) {
            delete[] pobj->pArray;
            pobj->pArray = nullptr;
        }

        //删除本身
        delete pobj;
        pobj = nullptr;
    }
}
```

```
double* FillArray(ArrayMaker* pobj, int num, double value)
{
    double* data = nullptr;

    if (pobj != nullptr) {
        data = pobj->pArray;
        if (data != nullptr) {
            delete[] data;
        }

        data = new double[num];
        for (int i = 0; i < num; i++) {
            data[i] = value;
        }
        pobj->pArray = data;
        pobj->nNumber = num;
    }

    return data;
}
```

1.2 C/C++ 调用 Julia

基于 Julia 原生API

- C/C++调用Julia，本质上C/C++调用Julia动态库，C/C++工程的编译和运行都依赖Julia及Julia仓库。
- C/C++调用Julia的配置方法，详细请参见：<https://juliacn.gitlab.io/JuliaZH.jl/manual/embedding.html>。

下例为函数展示了使用c++启动julia，并调用Julia的"print(sqrt(2.0))"语句

```
#include <julia.h>
JULIA_DEFINE_FAST_TLS // only define this once, in an executable (not in a shared library) if you want
fast code.

int main(int argc, char *argv[])
{
    /* required: setup the Julia context */
    jl_init();
    /* run Julia commands */
    jl_eval_string("print(sqrt(2.0))");
    /* strongly recommended: notify Julia that the
       program is about to terminate. this allows
       Julia time to cleanup pending write requests
       and run all finalizers
    */
    jl_atexit_hook(0); // 用户julia进程退出时执行的清理操作
    return 0;
}
```

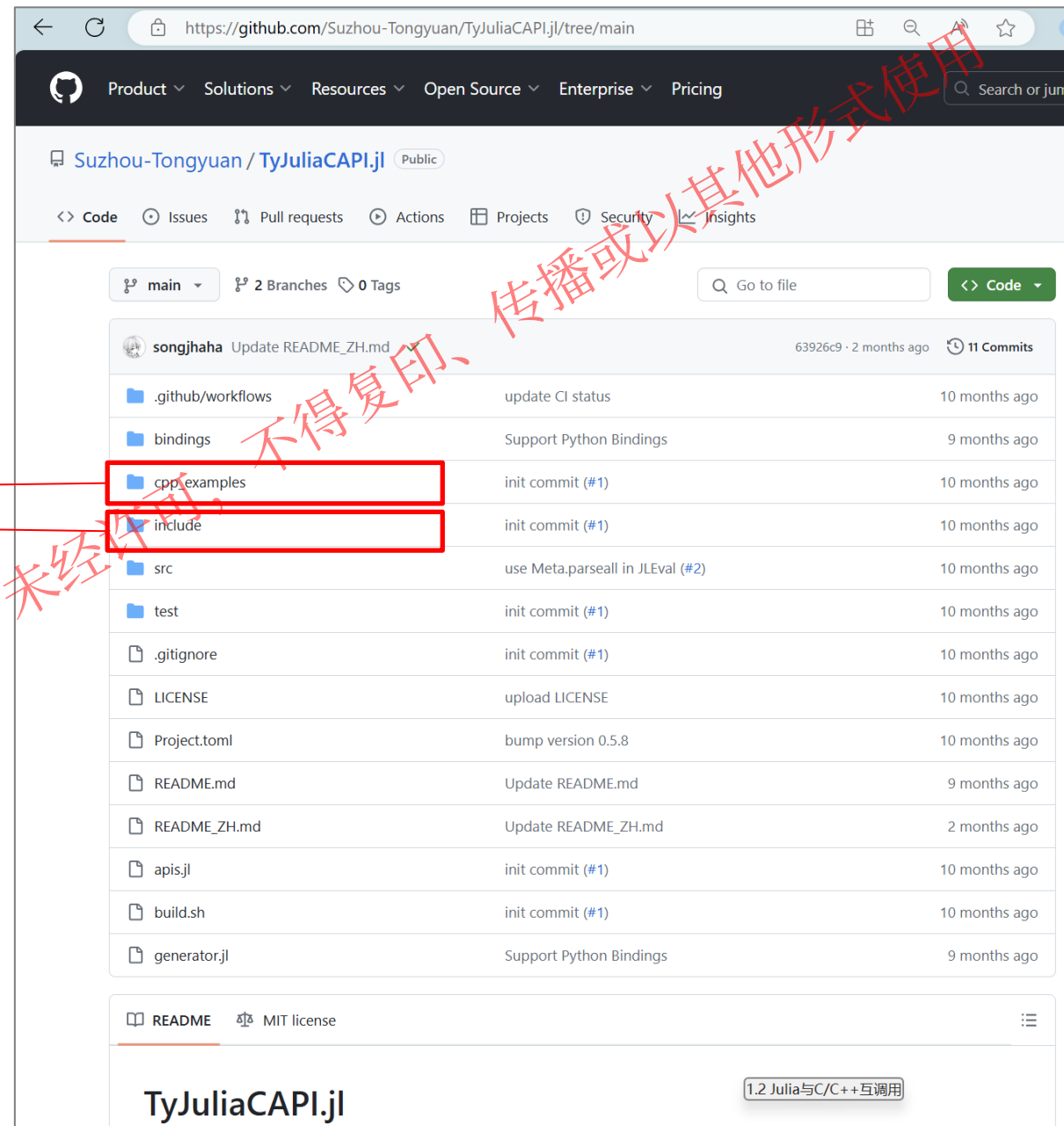
1.2 C/C++ 调用 Julia

C/C++调用Julia：基于同元的 TyJuliaCAPI

- TyJuliaCAPI提供一套完整的原子API，优点是高性能、易用的生命周期管理
- Syslab 已内置提供此包，该包源码地址：
<https://github.com/Suzhou-Tongyuan>

C++调用Julia的示例

C++接口源码



02

Julia与Python互调用

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

2.1 Julia 调用 Python

在数值计算领域，存在很多用Python写的高质量且成熟的库，为了便捷复用现有资产，Julia提供简洁且高效的调用方式，不需要任何“胶水”代码。

Julia调用python扩展库中函数：**pyimport** 和 **@pyimport**

```
# 依赖PyCall
using PyCall

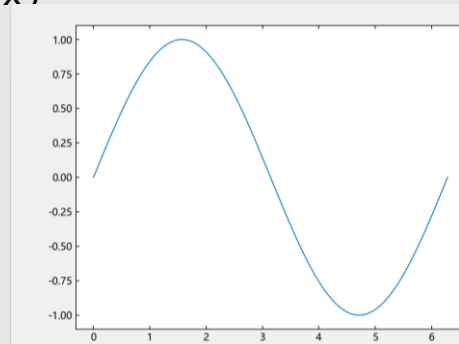
# 导入python库
math = pyimport("math")

v = math.sin(pi/2)
println("v = $v")
# v = 1.0
```

```
using PyCall
using TyPlot # 同元绘图库

# 导入python库
@pyimport numpy as np
x = np.linspace(0, 2pi, 1000)
y = np.sin(x)

# 绘图
plot(x,y)
```



目前Syslab安装包已经提供了完备的Julia和Python。

若用户想要使用自己的Python环境，需要参考Syslab帮助文档“Syslab-外部语言接口-Python调用Julia”

2.1 Julia 调用 Python

可以直接在Julia中嵌入python代码: **py**

```
using PyCall

# 直接嵌入python代码
py"""
str = "3 * 4 + 5"
a = compile(str, '', 'eval')
print("a =", eval(a))
"""

# a = 17
```

```
module MyModule
using PyCall

function __init__()
    py"""
    def hello(s):
        return "hello " + s
    """
end

# 直接调用python函数
hello(s) = py"hello"(s)

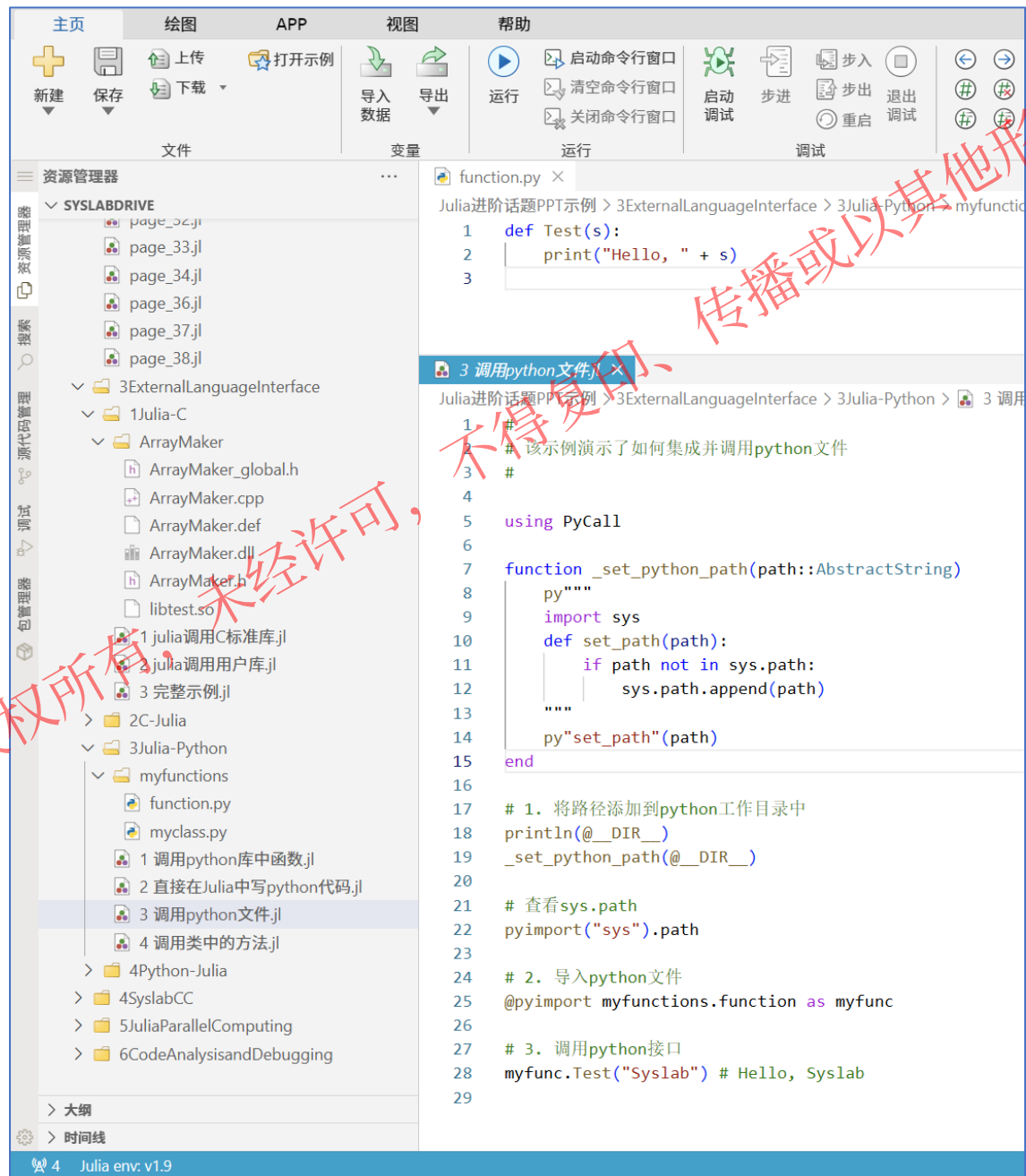
end # MyModule

MyModule.hello("julia")
# "hello julia"
```

2.1 Julia 调用 Python

调用python代码文件:

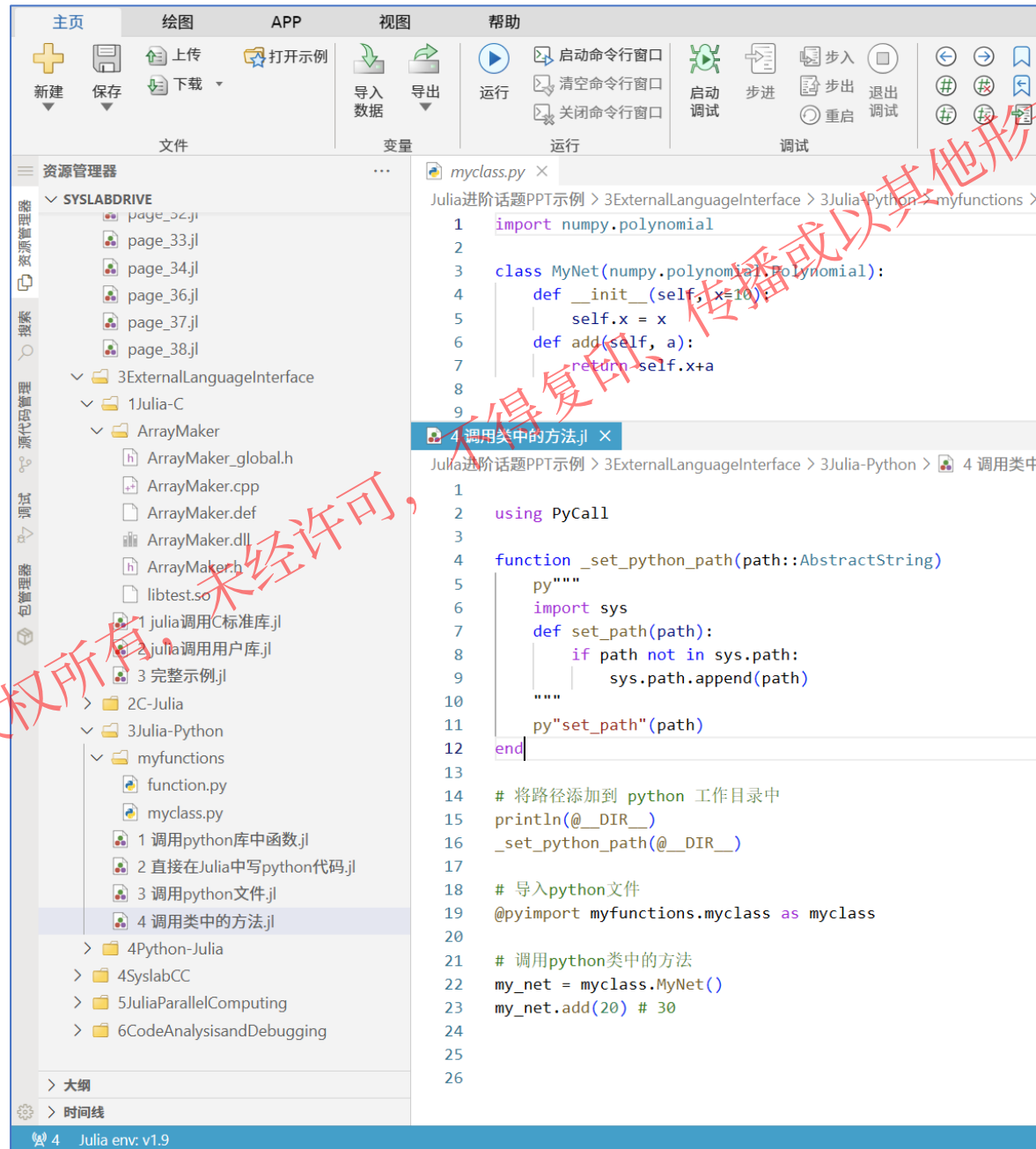
- ① 将路径添加到python工作目录中
- ② 导入python文件
- ③ 调用python接口



2.1 Julia 调用 Python

调用python类中的方法:

- ① 将路径添加到python工作目录中
- ② 导入python文件
- ③ 获取类对象
- ④ 调用python类中的方法



2.1 Julia 调用 Python

Julia调用Python过程中，支持Python回调Julia函数

资源管理器

SYSLABDRIVE

- page_33.jl
- page_34.jl
- page_36.jl
- page_37.jl
- page_38.jl
- 3ExternalLanguageInterface
 - 1Julia-C
 - ArrayMaker
 - ArrayMaker_global.h
 - ArrayMaker.cpp
 - ArrayMaker.def
 - ArrayMaker.dll
 - ArrayMaker.h
 - libtest.so
 - 1 julia调用C标准库.jl
 - 2 julia调用用户库.jl
 - 3 完整示例.jl
 - 2C-Julia
 - 3Julia-Python
 - myfunctions
 - __pycache__
 - function.py
 - myapp.py
 - myclass.py
 - 1 调用python库中函数.jl
 - 2 直接在Julia中写python代码.jl
 - 3 调用python文件.jl
 - 4 调用类中的方法.jl
 - 5 Python回调Julia函数.jl
 - 4Python-Julia
 - 1 内嵌Julia代码.py
 - 2 调用Julia库函数.py
 - 4SyslabCC
 - 5JuliaParallelComputing
 - 6CodeAnalysisandDebugging

5 Python回调Julia函数.jl

```
1 using Pycall
2
3 function _set_python_path(path::AbstractString)
4     py"""
5     import sys
6     def set_path(path):
7         if path not in sys.path:
8             sys.path.append(path)
9     """
10    py"set_path"(path)
11 end
12
13 function my_add(xs::AbstractArray)
14     return +(xs...)
15 end
16
17 function my_multiply(xs::AbstractArray)
18     return *(xs...)
19 end
20
21 function setupapp(app_module::PyObject)
22     py"""
23     def _setup_add(mod, jl_func):
24         mod.JuliaAPI.add = jl_func
25
26     def _setup_multiply(mod, jl_func):
27         mod.JuliaAPI.multiply = jl_func
28     """
29
30     py"_setup_add"(app_module, my_add)
31     py"_setup_multiply"(app_module, my_multiply)
32 end
33
34 _set_python_path(@_DIR_)
35
36 function run()
37     myapp = pyimport("myfunctions.myapp")
38     setupapp(myapp)
39     myapp.test()
40 end
41
```

往Python中注册Julia函数

myapp.py

Python接口类，具体函数实现由Julia提供

```
1 class JuliaAPI:
2     @staticmethod
3     def add(xs):
4         raise NotImplementedError
5
6     def multiply(xs):
7         raise NotImplementedError
8
9 def test():
10    print("add 执行结束，结果为: ", JuliaAPI.add([2,3,5]))
11    print("multiply 执行结束，结果为: ", JuliaAPI.multiply([2,3,5]))

```

2.2 Python 调用 Julia

Python调用Julia

Julia通过PyCall可以很方便地调用Python代码。反过来，Python通过**tyjuliacall**、**julia-numpy**等扩展包来实现对Julia的调用（注，上述扩展包都是同元公司开发的）。

其中，[tyjuliacall](#) 主要用来实现Python对Julia代码的调用，julia-numpy 作为tyjuliacall的底层支撑。

示例1-内嵌Julia代码

- 启动Syslab，新建Python文件
- 写入Python代码(如右图)
- 运行得到结果

```
from tyjuliacall import Main
from tyjuliacall import JuliaEvaluator

# 调用Julia代码
JuliaEvaluator[
    r"""mutable struct S
        x :: Int
        y :: Int
    end"""
]
S = JuliaEvaluator["S"]
s = S(1, 2)
print(s.x)
print(s.y)

# 修改字段
s.x = 10
s.y = 5
print(s.x)
print(s.y)
```

```
tongyuan@uid221-labid17733-prod-f75dcf7bb-kdncx:~/SyslabUser
Data/SyslabDrive/test/Python-julia$ /usr/local/share/TongYua
n/.julia/miniforge3/bin/python "/home/tongyuan/SyslabUserDat
a/SyslabDrive/test/Python-julia/01 内嵌julia代码.py"导入tyju
liacall: 2.69 s
1
2
10
5
```

2.2 Python 调用 Julia

示例2-调用Julia库函数

- 启动Syslab, 新建Python文件
- 写入Python代码(如右图)
- 运行得到结果

medfilt1 是Syslab信号处理工具箱函数

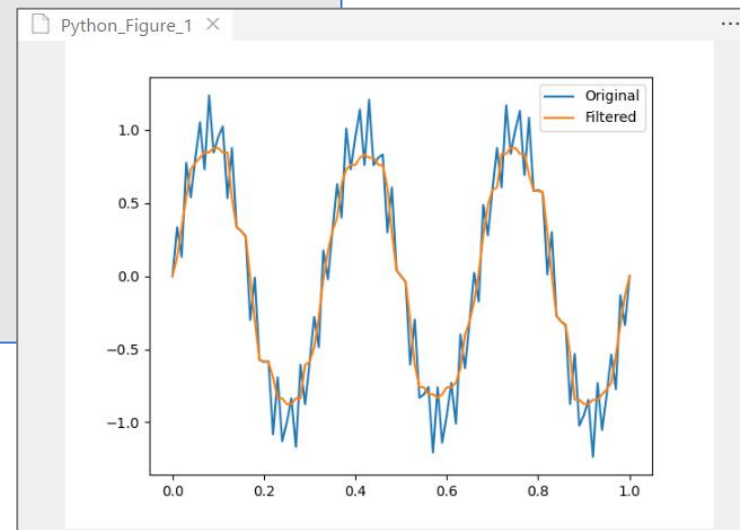
```
from mworks import TySignalProcessing

import numpy as np
from matplotlib import pyplot as plt

fs = 100
t = np.arange(fs + 1) / fs
print(t)
x = np.sin(2 * np.pi * t*3) + 0.25*np.sin(2 * np.pi * t*40)

# 调用信号库函数
y = TySignalProcessing.medfilt1(x, 9)

# 调用图形库函数
plt.plot(t, x, t, y)
plt.legend(np.asarray(["Original", "Filtered"]))
plt.show()
```



Syslab Julia 静态代码生成

Syslab 语言组

苏州同元软控信息技术有限公司

2024-10-22



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. 代码生成介绍

2. 代码生成产物

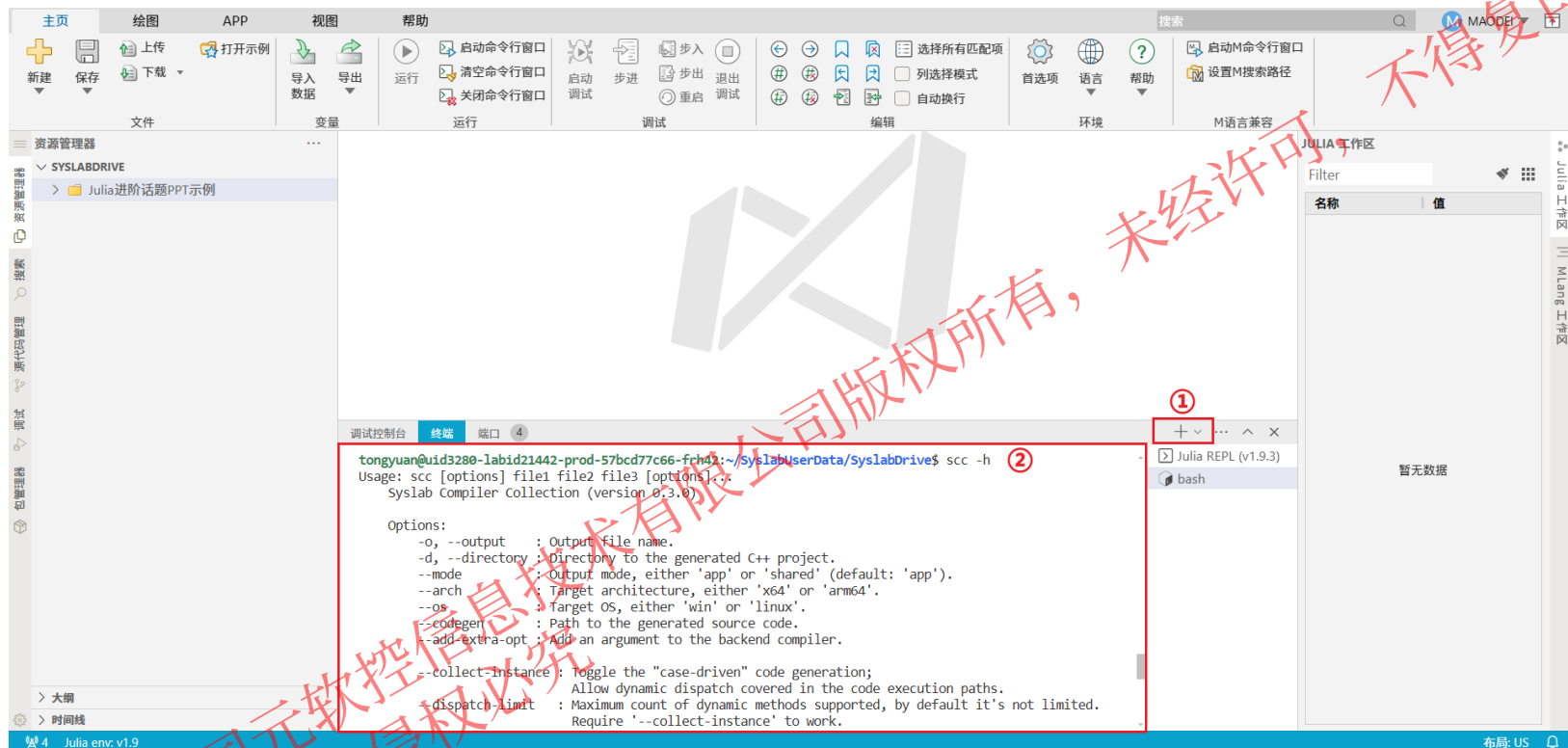
3. 代码生成工具选项

01 代码生成介绍

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1.代码生成介绍

Syslab 代码生成将 Julia 程序转换为可自行配置的 C++ 源码项目, 目前代码生成支持三种模式: 生成可执行文件、生成动态库、生成C++源码项目。Syslab终端中输入 `scc -h` 命令, 可输出代码生成工具scc的选项说明:



1.创建新终端, 选择不同终端环境

2.在新终端中访问 Syslab 代码生成工具 SCC

Windows 平台与 Linux 平台代码生成命令以及生成产物有差异, 具体参见Syslab帮助文档: 代码生成——Syslab 代码生成

02 代码生成产物

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

2.代码生成产物 - 可执行文件

用于生成可执行文件的 Julia 代码文件需要包含 `main()` 函数作为入口，将代码保存在`test.jl`文件中，在当前终端输入``scc test.jl -o main``命令即可生成当前平台（Linux）的可执行文件。

```
using TyMath

function main()
    A = 1:10000;
    A = reshape(A, 100, 100)
    display(fft(A)[1, 1:2])
end

@static @isdefined(SyslabCC) ||
main()
```



```
test.jl x
Julia进阶话题PPT示例 > 4SyslabCC > page_65 > test.jl
1 using TyMath
2 function main()
3     A = 1:10000
4     A = reshape(A, 100, 100)
5     display(fft(A)[1, 1:2])
6 end
7 @static @isdefined(SyslabCC) || main()
8

调试控制台 终端 端口 4
tongyuan@uid3280-labid21442-prod-57bcd77c66-frh42:~/SyslabUserData/SyslabDrive/Julia进阶话题PPT示例/4SyslabCC/page_65$ scc test.jl -o main
building apps...
mkdir -p bin
mkdir -p lib
g++ -std=c++11 -O2 -Wno-unused-variable -Wno-unused-value -o bin/main main.cpp -lsyslabcr
rt-intrinsics -lsyslabcr-io/include -Ighc-filesystem -Lbin -lsyslabcr-dylib -lsyslabcr
t-io -lbdwgc -Wl,--as-needed -Wl,-rpath,'$ORIGIN/lib:$ORIGIN:.' -ldl -lpthread
rm -rf bin/lib
cp -rf lib bin/
[Info]: Build succeeded.
tongyuan@uid3280-labid21442-prod-57bcd77c66-frh42:~/SyslabUserData/SyslabDrive/Julia进阶话题PPT示例/4SyslabCC/page_65$ ./main
2-element Vector{ComplexF64}
5.001e7 + 0.0im
-500000.0 + 1.591e7im
```

`scc test.jl -o main`

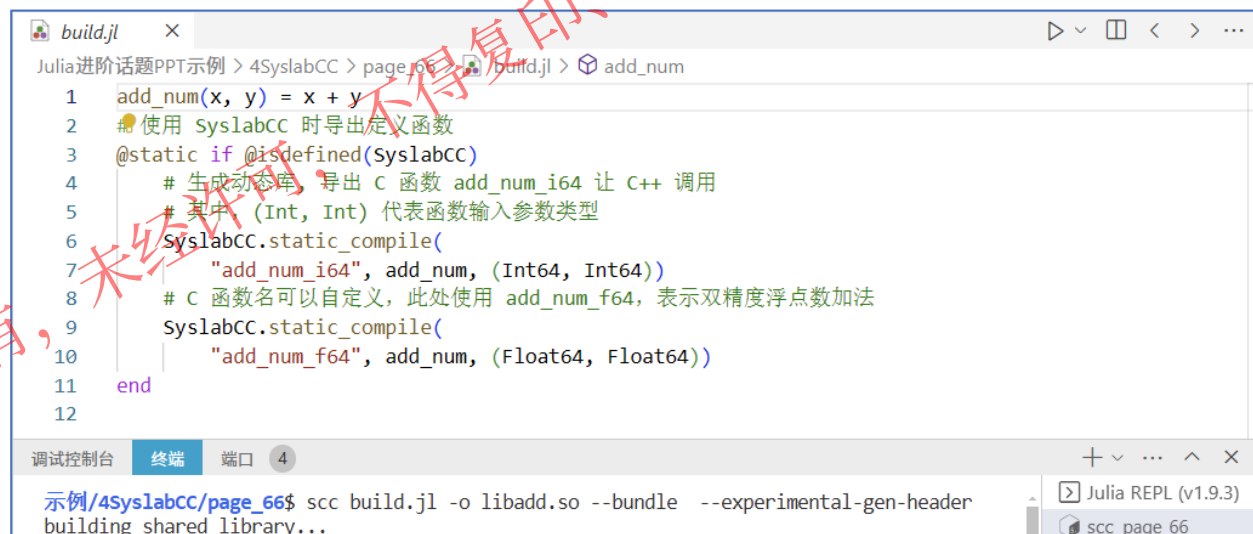
2.代码生成产物 - 动态库

SyslabCC 支持将 Julia 编写的函数导出给外部语言调用，且调用方无需安装 Syslab。此功能可通过 SyslabCC 的动态库生成能力来满足。下面演示为 add_num 函数创建动态库，并在 C++ 中调用此函数。

```
add_num(x, y) = x + y

# 使用 SyslabCC 时导出定义函数
@static if @isdefined(SyslabCC)
    # 生成动态库，导出 C 函数
    add_num_i64 让 C++ 调用
    # 其中，(Int, Int) 代表函数输入参数类型
    SyslabCC.static_compile(
        "add_num_i64", add_num,
        (Int64, Int64));

    # C 函数名可以自定义，此处使用
    add_num_f64，表示双精度浮点数加法
    SyslabCC.static_compile(
        "add_num_f64", add_num,
        (Float64, Float64));
end
```



```
build.jl
Julia进阶话题PPT示例 > 4SyslabCC > page_66 > build.jl > add_num
1 add_num(x, y) = x + y
2 使用 SyslabCC 时导出定义函数
3 @static if @isdefined(SyslabCC)
4     # 生成动态库，导出 C 函数 add_num_i64 让 C++ 调用
5     # 其中 (Int, Int) 代表函数输入参数类型
6     SyslabCC.static_compile(
7         "add_num_i64", add_num, (Int64, Int64))
8     # C 函数名可以自定义，此处使用 add_num_f64，表示双精度浮点数加法
9     SyslabCC.static_compile(
10        "add_num_f64", add_num, (Float64, Float64))
11 end
12

调试控制台 终端 端口 4
示例/4SyslabCC/page_66$ scc build.jl -o libadd.so --bundle --experimental-gen-header
building shared library...
```

scc build.jl -o libadd.so --bundle --experimental-gen-header

--experimental-gen-header选项可自动生成头文件，给C++程序include

生成的产物主要有：libbdwgc.so、libadd.h、libadd.so

2.代码生成产物 - 动态库

接下来，使用以下 test_add_num.cpp 文件，调用生成动态库中的函数 add_num_i64 和 add_num_f64，以验证由 build.jl 文件中 Julia 代码生成的动态库 libadd.so（Linux 系统下）。

```
#include <iostream>
#include <stdint.h>
#include "libadd.h"

int main()
{
    double af = 1.2;
    double bf = 2.4;
    std::cout << "add_num_f64(a = 1.2, b = 2.4) = " << add_num_f64(af, bf) << std::endl;

    int64_t a1 = 2;
    int64_t b1 = 4;
    std::cout << "add_num_i64(a = 2, b = 4) = " << add_num_i64(a1, b1) << std::endl;

    return 0;
}
```



The screenshot shows a code editor with the file test_add_num.cpp. The code is identical to the one in the previous block. Below the editor is a terminal window showing the compilation and execution of the program. The terminal output is as follows:

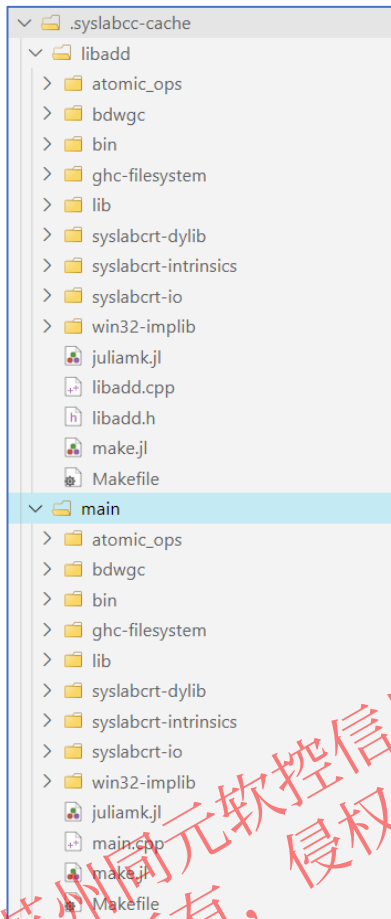
```
tongyuan@uid3280-labid21442-prod-57bcd77c66-frh42:~/SyslabUserData/SyslabDrive/Julia进阶话题PPT示例/4SyslabCC/page_66_67$ g++ test_add_num.cpp -o test -I"." -L"." -l:./libadd.so -lpthread -std=c++11
tongyuan@uid3280-labid21442-prod-57bcd77c66-frh42:~/SyslabUserData/SyslabDrive/Julia进阶话题PPT示例/4SyslabCC/page_66_67$ ./test
add_num_f64(a = 1.2, b = 2.4) = 3.6
add_num_i64(a = 2, b = 4) = 6
```

以 g++ 编译器为例来演示，不同编译器的命令可能存在差异

g++ test_add_num.cpp -o test -I"." -L"." -l:./libadd.so -lpthread -std=c++11

2.代码生成产物 – C++项目

C++项目生成是 Syslab代码生成工具提供的核心能力。当代码生成输出可执行文件或动态库时，在当前工作目录的 .syslabcc-cache 目录下会同步创建对应名称的 C++ 项目。



Syslab 代码生成的一个关键功能是生成 C++ 项目到指定位置，但不立刻编译它。以之前 libadd 动态库为例，如使用 `scc build.jl -d myproj -c --mode shared` 命令，Syslab代码生成工具会生成 C++ 项目到 myproj 目录，但不是立刻编译它。

目录中的 Makefile 文件即为 make 构建脚本，可以使用 make 工具自行构建。只需运行 `cd myproj` 命令进入 myproj 目录后，再运行 `make all -j 8` 即可完成构建。作为备选方案，进入 myproj 目录后，也可以运行 `julia make.jl all` 来构建项目。

```
(base) PS C:\Users\TR\Test> cd myproj
(base) PS C:\Users\TR\Test\myproj> julia make.jl all
`zig cc -target x86_64-windows-gnu -fPIC -c -w -O2 -fno-sanitize=undefined -fno-stack-protector
-Isyslabcrt-io/include -o syslabcrt-io/src/ios.o syslabcrt-io/src/ios.c`
`zig cc -target x86_64-windows-gnu -fPIC -c -w -O2 -fno-sanitize=undefined -fno-stack-protector
-Isyslabcrt-io/include -o syslabcrt-io/src/utf8.o syslabcrt-io/src/utf8.c`
`zig cc -target x86_64-windows-gnu -fPIC -c -w -O2 -fno-sanitize=undefined -fno-stack-protector
-Isyslabcrt-io/include -o syslabcrt-io/src/utills.o syslabcrt-io/src/utills.c`
`zig cc -target x86_64-windows-gnu -fPIC -c -w -O2 -fno-sanitize=undefined -fno-stack-protector
-Isyslabcrt-io/include -o syslabcrt-io/src/utf8proc/utf8proc.o syslabcrt-io/src/utf8proc/utf8p
roc.c`
```

03 代码生成工具选项

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

3.代码生成工具选项

-o {filename} 指定编译输出文件名

--mode {app | shared}指定编译模式，默认为 app 模式生成可执行文件，指定为 --mode shared 生成动态库

-d {path} 指定C++项目输出目录

--bundle 打包自包含版本的代码生成产物，动态库依赖将被自动收集到 lib 目录中，便于应用部署

--codegen {path} 指定生成的 C++ 源码路径

--c 或 --no-compile 只生成C++项目而不进行后续编译过程

--force-juliamk 强制使用juliamk构建系统

--experimental-gen-header 为动态库自动生成头文件

--full-aot 将动态调用视为编译器错误，目前仅支持 ' ALL' 参数

3.代码生成工具选项

--arch {x86 | arm64} 指定目标架构，用于交叉编译功能

--os {win | linux} 指定目标操作系统，用于交叉编译功能

--no-blas 初始化时不加载blas动态库，交叉编译需要开启

--collect-instance 启动用例驱动的代码生成，搜集代码生成过程中的动态分派

--dispatch-limit {number}指定最大动态派发方法的数量，需要开启 --collect-instance

--static-mingw 静态链接MinGW运行时，避免Windows程序再其他机器上缺少动态链接库的问题

--verbose 打印详情信息

--debug 打印开发者 debug 信息

--debugtrace 当运行时错误发生时显示 Julia 源码调用堆栈

Syslab Julia 并行计算

Syslab 语言组

苏州同元软控信息技术有限公司

2024-10-24



TONGYUAN
Software and Control

目录

1. Julia 异步编程

2. Julia 多线程编程

01 Julia 异步编程

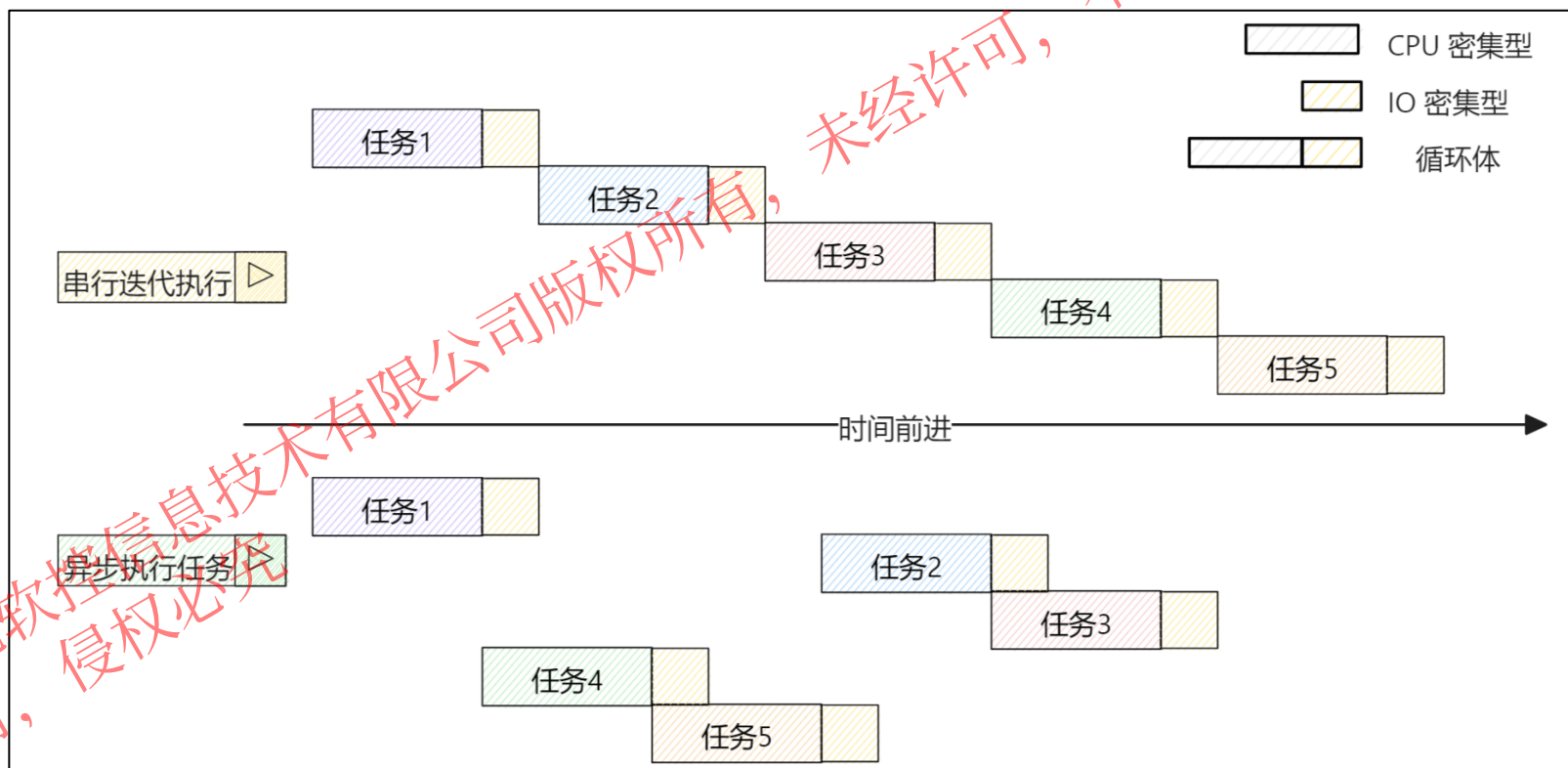
©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

1. Julia 异步编程

为什么需要异步

普通的函数大多是串行执行的，现代计算机系统常常将计算模块和IO模块（如硬盘读写，网络请求和GPU计算）拆分，这意味着IO操作背后使用的是独立的硬件资源。因此，对于相互独立的任务来说，我们很容易会产生一个想法：如果任务使用的硬件资源存在不可忽略的耗时，那么在使用这些硬件资源时我们可以尝试完成其他CPU计算。例如：发送网络请求后等待回应的这段时间并不妨碍CPU进行其他计算任务。下图就是对此想法的直观描述：



1. Julia 异步编程

Julia 任务 (Task)

在 Julia 中，任何一个无参函数都可以被封装为一个任务 (Task)，其中无参函数指的是可以以 `f()` 形式被调用的函数或对象。

```
f() = println("Hello")
```

可以通过将无参函数传给 Task 创建一个任务：

```
t = Task(f)
```

也可以通过 `@task` 宏创建一个任务

```
t = @task println("Hello")
```

接下来我们将介绍两种典型的任务：CPU密集型任务与IO密集型任务。进一步地，我们将介绍如何使用异步的方式来调度这些任务。

1. Julia 异步编程

CPU 密集型任务

下面我们使用busywait函数模拟一个 CPU 密集型计算，然后通过 Task 为该计算创建一个任务。

```
function busywait(seconds=5)
    # 该函数本身没有实际意义
    # 在这里仅仅是构造一个一直占用 CPU 的定时函数，用于演示异步任务的调度
    tstart =time_ns()
    while (time_ns() - tstart) /1e9< seconds
        continue
    end
    return (time_ns() - tstart) /1e9
end
t = Task(busywait)
```

为了让Task任务开始运行，我们需要通过schedule函数将任务加入调度队列：

```
schedule(t)
```

此时我们会观察到一个 5s 左右的停顿，此时Julia正在不断地重复 busywait 内的 while 循环，我们可以粗略的认为该任务占用了 5s 的 CPU 时间。

1. Julia 异步编程

IO 密集型任务

IO 密集型任务指的是，如：文件读写、网络请求、数据库查询等等。sleep函数的效果是让任务等待一段时间，在等待的这段时间中将不占用CPU时间，在这里我们使用 Julia 标准库的 sleep 函数来模拟一个 IO 密集型计算。

```
sleep(5) # 让当前任务等待 5 秒
```

通过 @task 创建一个IO密集型任务

```
t = @task sleep(5)
```

通过schedule将该任务加入调度，由于等待的时间不占用CPU时间，schedule函数在将任务加入调度后立即返回，此刻用户可以继续进行其他计算。

```
schedule(t) # 立即返回 (但任务还没执行完)
```

1. Julia 异步编程

通过并发提升计算效率

通过有效的组织CPU密集型任务和IO密集型任务可以提升计算效率，f_sync 函数将依次串行执行计算CPU密集型任务和IO密集型任务共5次：

```
function f_sync()  
    for i in 1:5  
        busywait(2) # 时长 2 秒的计算密集型任务  
        sleep(1)    # 时长 1 秒的IO密集型任务  
    end  
end
```

统计运行一次f_sync的总时长，大致为所有任务运行时长的综合：

```
@time f_sync()
```

```
15.046421 seconds (25 allocations: 720 bytes)
```

1. Julia 异步编程

通过并发提升计算效率

以下为并发版本的程序，其中 `@async expr` 的作用等价于 `schedule(@task expr)`，即创建一个任务并将其加入调度，`@sync expr` 在这里则表示等待`expr`中所有通过`@async`启动的任务全部执行完毕。

```
function f_async()
    @sync begin
        for i in 1:5
            @async busywait(2) # 时长 2 秒的CPU密集型任务
            @async sleep(1)    # 时长 1 秒的IO密集型任务
        end
    end
end
```

统计运行一次`f_async`的时长，总时长相比于串行执行版本减少了大约4秒，这是因为在前4次迭代中，无需等待 `sleep(1)` 运行结束即可进入下一次迭代，而串行执行必须等待`sleep`运行结束才能进入下一次迭代。

```
@time f_async()
```

```
11.002561 seconds (76 allocations: 6.094 KiB)
```


02 Julia 多线程编程

未经授权，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

2. Julia 多线程编程

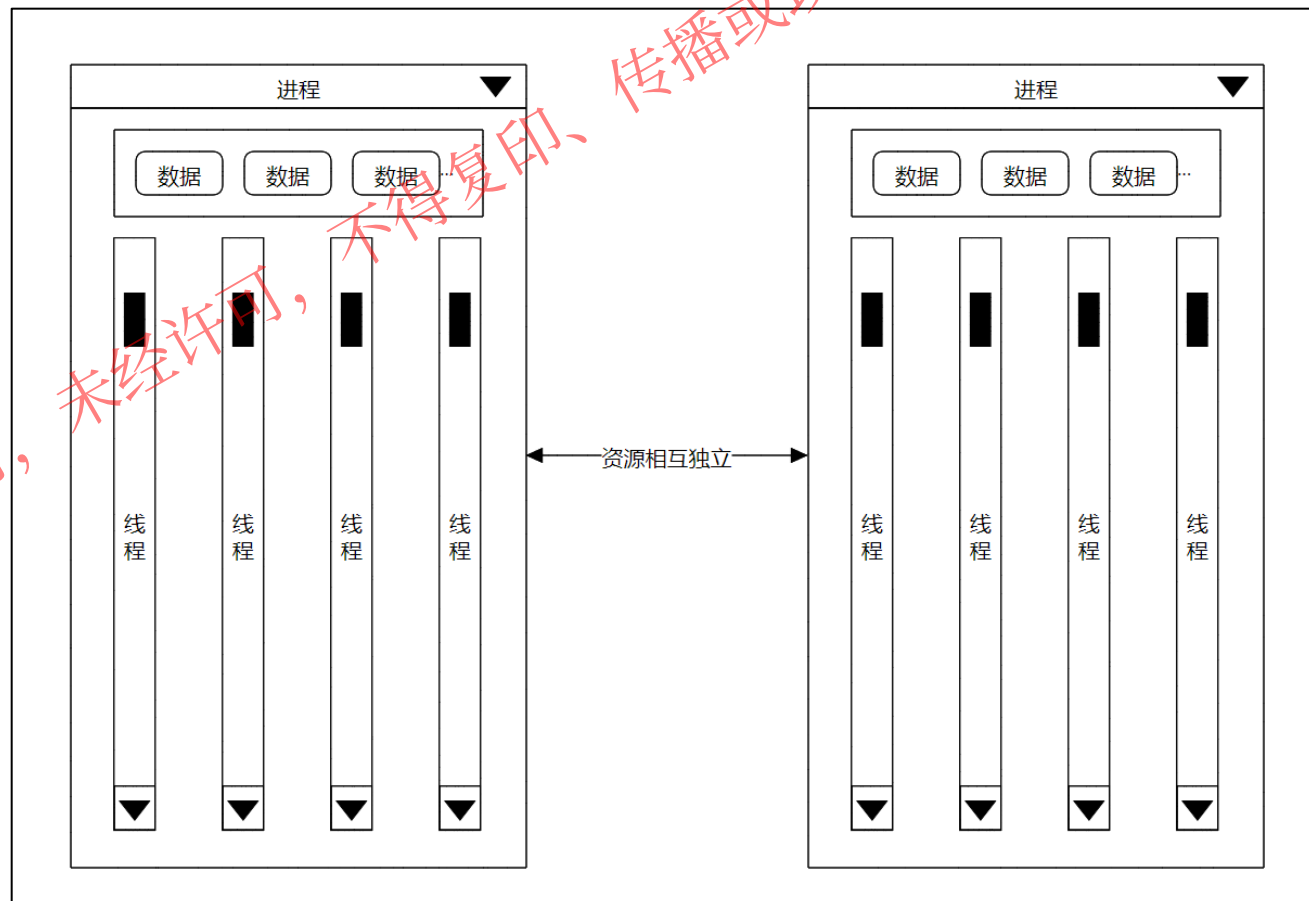
多线程和异步的区别

在异步中，我们学会了如何通过异步同时运行需求资源不相关的任务，对于IO和计算任务，这能提高硬件资源利用率并节省执行时间。但仅靠异步本身并不能调动多个CPU核心——异步程序的控制流仍是由单个CPU调度执行的。

调度更多的 CPU 资源的策略一般分为**多线程**或**多进程**

。多线程是在同一Julia进程中创建多个线程，并通过共享内存的方式访问数据。多进程则是创建多个Julia程序，然后通过进程间通信的方式交换数据。

进程间资源是相互独立的，但同一进程的各线程间会共享进程的资源。本节课程中仅介绍Julia多线程的使用方式。

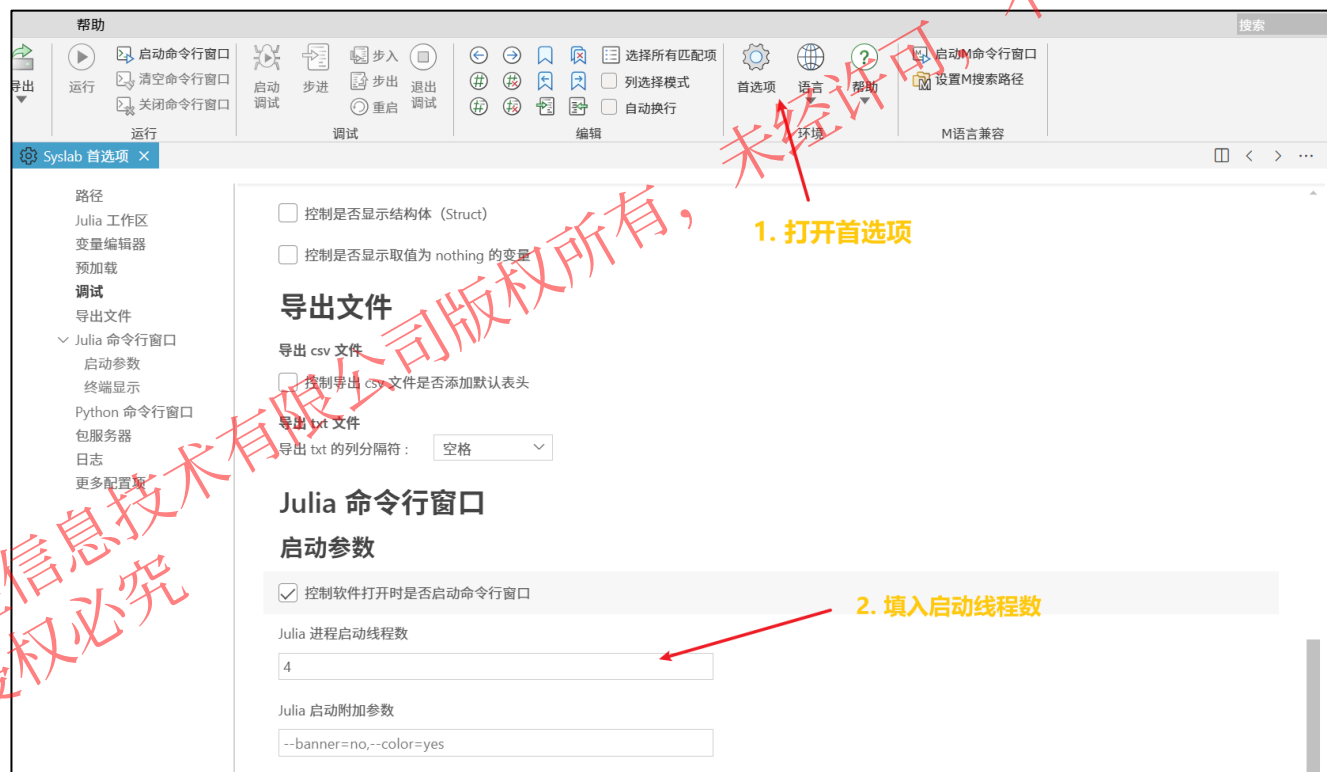


2. Julia 多线程编程

启用Julia 多线程

为启用 Julia 多线程，我们需要在命令行通过 `-t` 选项指定 Julia 线程的数量，也可以在 Syslab 首选项中设置 Julia 进程的线程启动数，例如，我们通过 `julia -t 4` 启动一个具有4个线程的Julia进程

```
julia -t 4
```



2. Julia 多线程编程

启用Julia 多线程

为了使用多线程相关的函数或宏，我们需要加载 Base.Threads 模块：

```
using Base.Threads
```

启用多线程后，我们即可通过nthread函数查看线程数目：

```
nthreads()
```

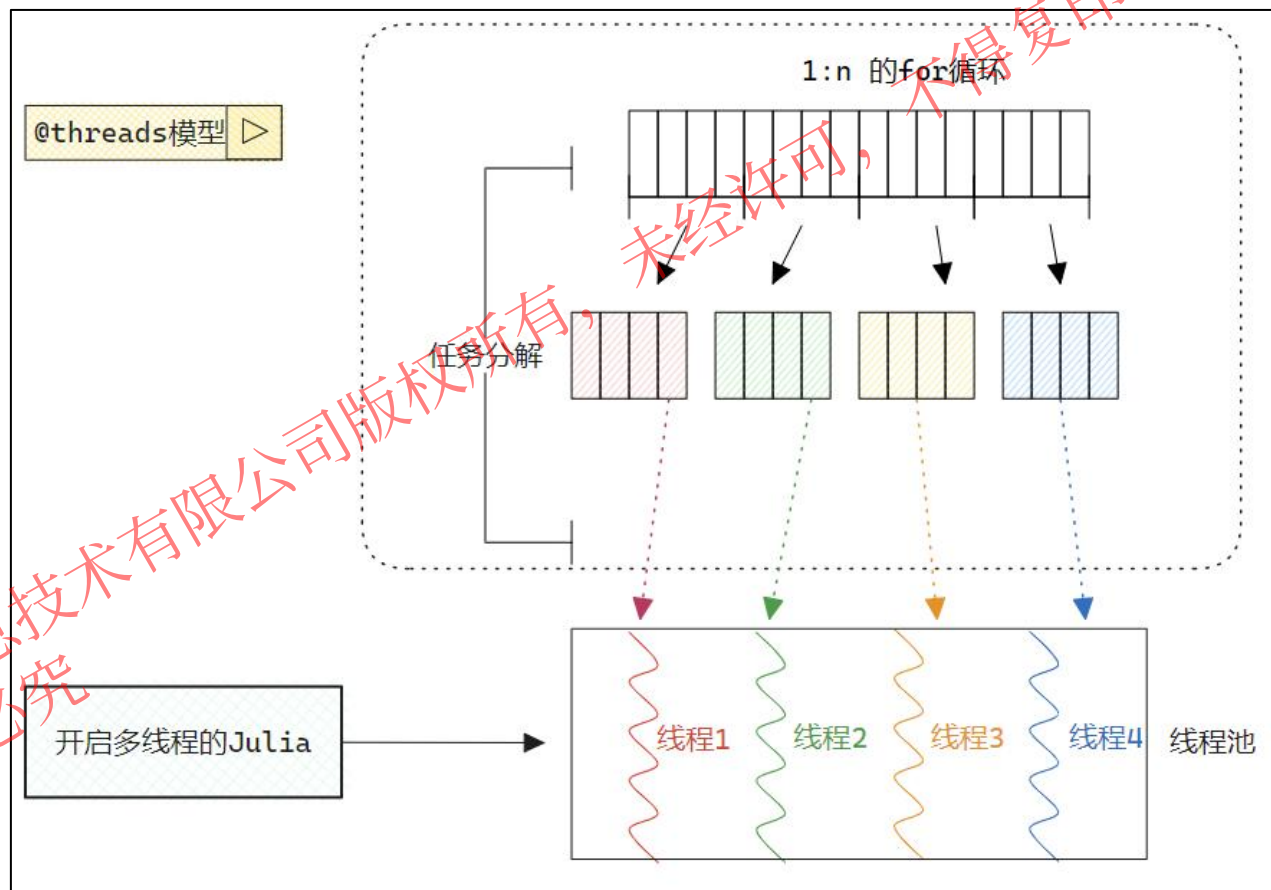
我们可以通过threadid查看当前线程ID，在未创建线程之前，Julia运行在线程ID为1的主线程上：

```
threadid()
```

2. Julia 多线程编程

@threads 并行 for 循环

在for循环前加入@threads后，julia会将for循环体中的计算任务均匀划分后分配给每个线程，@threads for 不保证for循环体的执行顺序，因此使用@threads for时要求每个由for循环体创建的计算任务是顺序无关的。下图展示了@threads for的编程模型。



2. Julia 多线程编程

@threads 并行 for 循环

通过@thread for并行执行了10次循环体，我们可以看到任务均匀划分在各个线程上运行，从而可以看出循环语句在各线程中并行地执行。

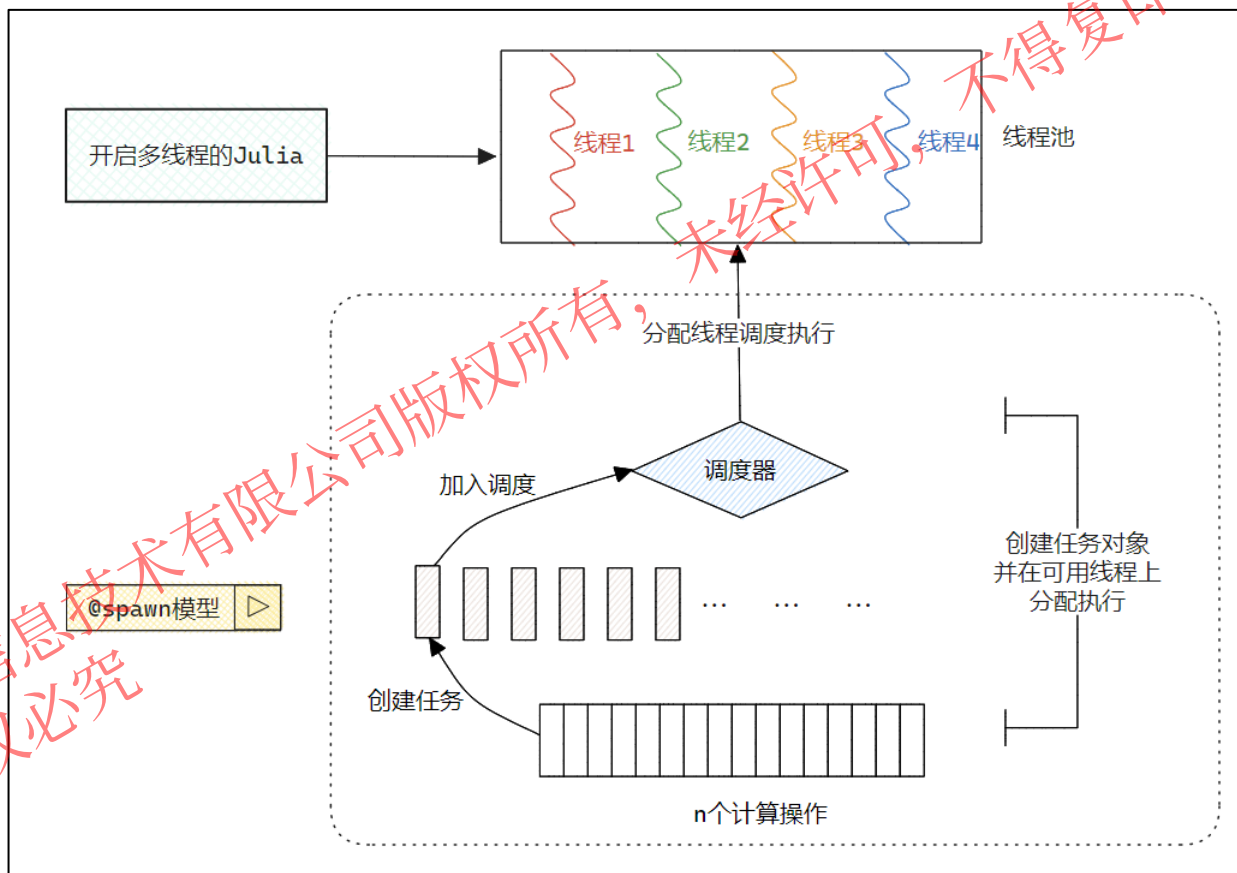
```
@time @threads for i in 1:10
    sleep(1)
    @show i, threadid()
end
```

```
(i, threadid()) = (9, 4)
(i, threadid()) = (7, 3)
(i, threadid()) = (4, 2)
(i, threadid()) = (1, 1)
(i, threadid()) = (10, 4)
(i, threadid()) = (8, 3)
(i, threadid()) = (2, 1)
(i, threadid()) = (5, 2)
(i, threadid()) = (6, 2)
(i, threadid()) = (3, 1)
3.095430 seconds (143.44 k allocations: 8.086 MiB, 2.04% compilation time)
```

2. Julia 多线程编程

@spawn 创建任务并运行在可用线程上

与异步的@async类似，在多线程中，Threads.@spawn也会将计算任务创建为Task任务对象并立即执行。与@async不同的是，Threads.@spawn创建并调度的任务会在任何可用线程上调度执行，而@async创建的任务只会在主线程上执行。下图展示了Julia @spawn的编程模型：



2. Julia 多线程编程

@spawn 创建任务并运行在可用线程上

我们通过 @spawn 实现了之前用 @thread for 实现的功能，其中 @sync 会等待所有for循环中的任务执行结束后退出：

```
@time @sync for i in 1:10
    Threads.@spawn begin
        sleep(1)
        @show i, threadid()
    end
end
```

```
(i, threadid()) = (7, 4)
(i, threadid()) = (5, 3)
(i, threadid()) = (3, 2)
(i, threadid()) = (10, 4)
...
1.026771 seconds (9.33 k allocations: 541.373 KiB, 2.05% compilation time)
```


2. Julia 多线程编程

数据竞争问题

正如一开始我们提到的，多线程中的内存数据是共享的，因此若多个线程同时对同一内存进行读写，就会出现数据竞争问题。

```
function mysum_threads(A)
    rst = 0
    @threads for i in 1:length(A)
        rst += A[i]
    end
    return rst
end
```

在上面的代码中，我们尝试多线程的计算数组的累加结果，但是由于多个线程对 `rst` 访问存在数据竞争，因此启用多线程时多线程后，每次执行 `mysum_threads(1:100_000)` 都有可能得到不一样的结果。

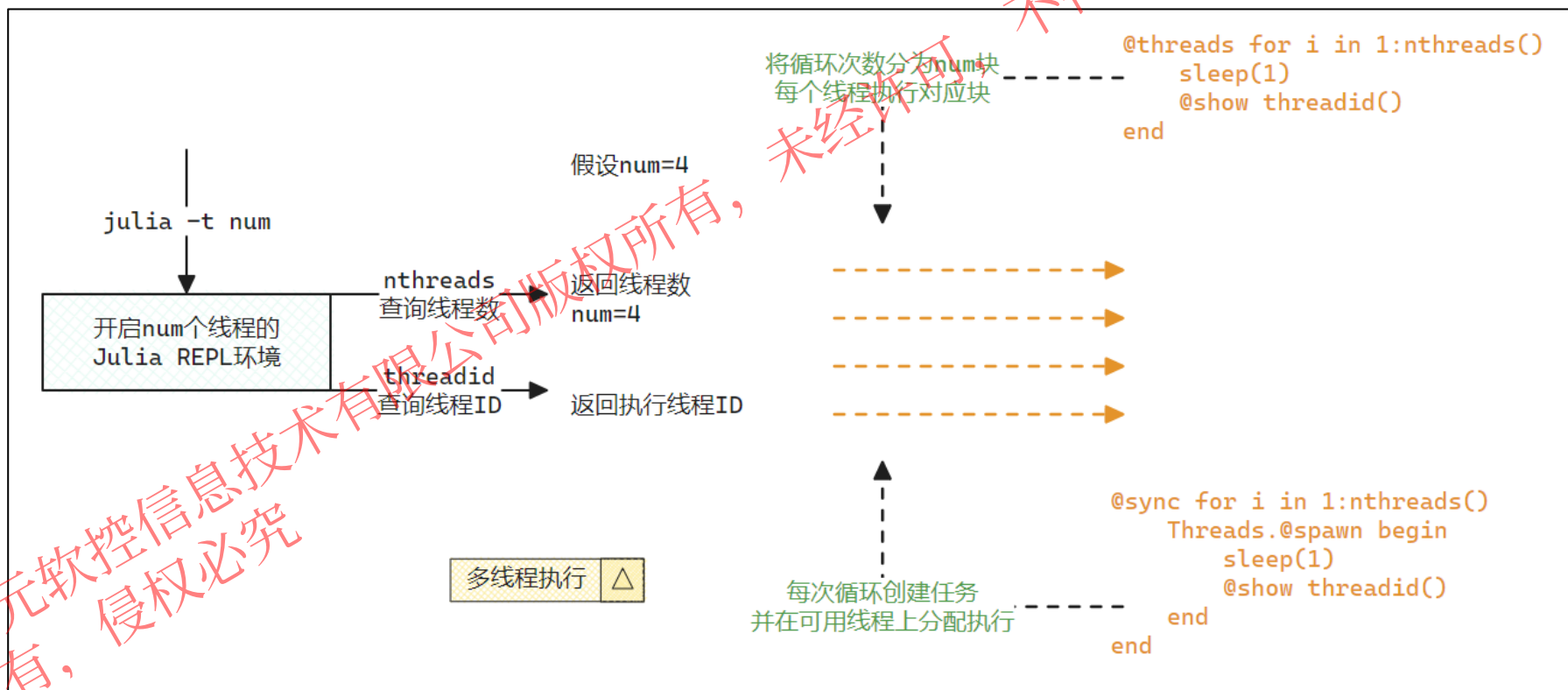
关于数据竞争的进一步介绍以及它们的处理方式，可以参考Syslab帮助文档的《多线程数据竞争》章节。

2. Julia 多线程编程

小结

本章中我们学习了以下内容：

1. 多线程支持需要通过 `-t` 或 Syslab首选项开启
2. 常用的多线程计算的API有 `@thread for` 和 `@spawn`
3. 多线程的计算可能会产生数据竞争问题



Syslab 代码分析与调试

Syslab 语言组

苏州同元软控信息技术有限公司

2024-10-26



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，侵权必究
未经许可，不得复印、传播或以其他方式使用

目录

1. 程序性能分析

2. 代码动态更新

01 程序性能分析

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

1. 程序性能分析 – 程序计时

@time

@time 宏对一个表达式计时和查看内存分配情况，包括：

- 程序运行时间
- 内存分配次数
- 总计分配的内存数
- 垃圾回收时间占比 (gc time)
- 编译代码时间占比

```
@time expression
```

- expression 是你要测量执行时间和内存分配的代码块。

以下是使用 @time 的示例：

- 例1：测量简单表达式的执行时间

```
x = rand(10, 10);  
  
@time x * x; # 第一次存在编译时间  
  
@time x * x;
```



```
julia> x = rand(10, 10);  
  
julia> @time x * x; # 第一次存在编译时间  
0.701538 seconds (1.38 M allocations: 94.059 MiB, 6.14% gc time,  
99.94% compilation time)  
  
julia> @time x * x;  
0.000013 seconds (1 allocation: 896 bytes)
```

1. 程序性能分析 – 程序计时

@elapsed

@elapsed 宏对表达式计时，以浮点数的形式返回执行所用的秒数。

```
time_taken = @elapsed expression
```

- expression 是要测量执行时间的代码块。

以下是使用 @elapsed 的示例：

- 例1：测量简单表达式的执行时间

```
x = rand(10, 10);  
  
time_taken = @elapsed x * x
```

```
julia> x = rand(10, 10);  
  
julia> time_taken = @elapsed x * x  
1.1194e-5
```

- 例2：测量函数调用的执行时间

```
function compute_sum(n)  
    num = 0  
    for i in 1:n  
        num += i  
    end  
    return num  
end  
time_taken = @elapsed compute_sum(1000000)
```

```
julia> function compute_sum(n)  
    num = 0  
    for i in 1:n  
        num += i  
    end  
    return num  
end  
e_taken = @elapsed compute_sum(1000000)  
compute_sum (generic function with 1 method)  
  
julia> time_taken = @elapsed compute_sum(1000000)  
1.4e-7
```

1. 程序性能分析 – 程序计时

@btime

@btime 是 Julia 中 BenchmarkTools 包提供的一个宏，提供在基准测试期间测量的最小运行时间。

```
@btime expression
```

- expression 是要测量执行时间的代码块

以下是使用 @btime 的示例：

- 例1：测量简单循环的执行时间

```
using BenchmarkTools
x = rand(10, 10);

@btime x * x;
```

- 例2：测量函数调用的执行时间

```
using BenchmarkTools
function compute_sum(n)
    num = 0
    for i in 1:n
        num += i
    end
    return num
end
@btime compute_sum(1000000);
```

```
julia> using BenchmarkTools
julia> x = rand(10, 10);
julia> @btime x * x;
278.910 ns (1 allocation: 896 bytes)
```

```
julia> using BenchmarkTools
julia> function compute_sum(n)
        num = 0
        for i in 1:n
            num += i
        end
        return num
    end
compute_sum (generic function with 1 method)
julia> @btime compute_sum(1000000);
2.847 ns (0 allocations: 0 bytes)
```


1. 程序性能分析 – 程序计时

@benchmark

@benchmark 是 Julia 中 BenchmarkTools 包提供的一个宏，用于对 Julia 表达式进行基准测试。

```
@benchmark expression
```

- expression 是要测量执行时间的代码块。

以下是使用 @benchmark 的示例：

- 例1：测量简单表达式的执行时间

```
using BenchmarkTools
x = rand(10, 10);

@benchmark x * x
```

- 例2：测量函数调用的执行时间

```
using BenchmarkTools
function compute_sum(n)
    num = 0
    for i in 1:n
        num += i
    end
    return num
end
@benchmark compute_sum(1000000)
```

```
julia> @benchmark x * x
BenchmarkTools.Trial: 10000 samples with 310 evaluations.
Range (min ... max): 268.281 ns ... 5.186 μs
Time (median): 287.850 ns
Time (mean ± σ): 328.751 ns ± 267.142 ns
GC (min ... max): 0.00% ... 85.98%
GC (median): 0.00%
GC (mean ± σ): 5.39% ± 6.36%

Histogram: log(frequency) by time
268 ns 851 ns <

Memory estimate: 896 bytes, allocs estimate: 1.
```

```
julia> @benchmark compute_sum(1000000)
BenchmarkTools.Trial: 10000 samples with 1000 evaluations.
Range (min ... max): 2.847 ns ... 16.318 ns
Time (median): 2.849 ns
Time (mean ± σ): 2.886 ns ± 0.507 ns
GC (min ... max): 0.00% ... 0.00%
GC (median): 0.00%
GC (mean ± σ): 0.00% ± 0.00%

Histogram: log(frequency) by time
2.85 ns 2.87 ns <

Memory estimate: 0 bytes, allocs estimate: 0.
```

1. 程序性能分析 – 程序性能采样分析

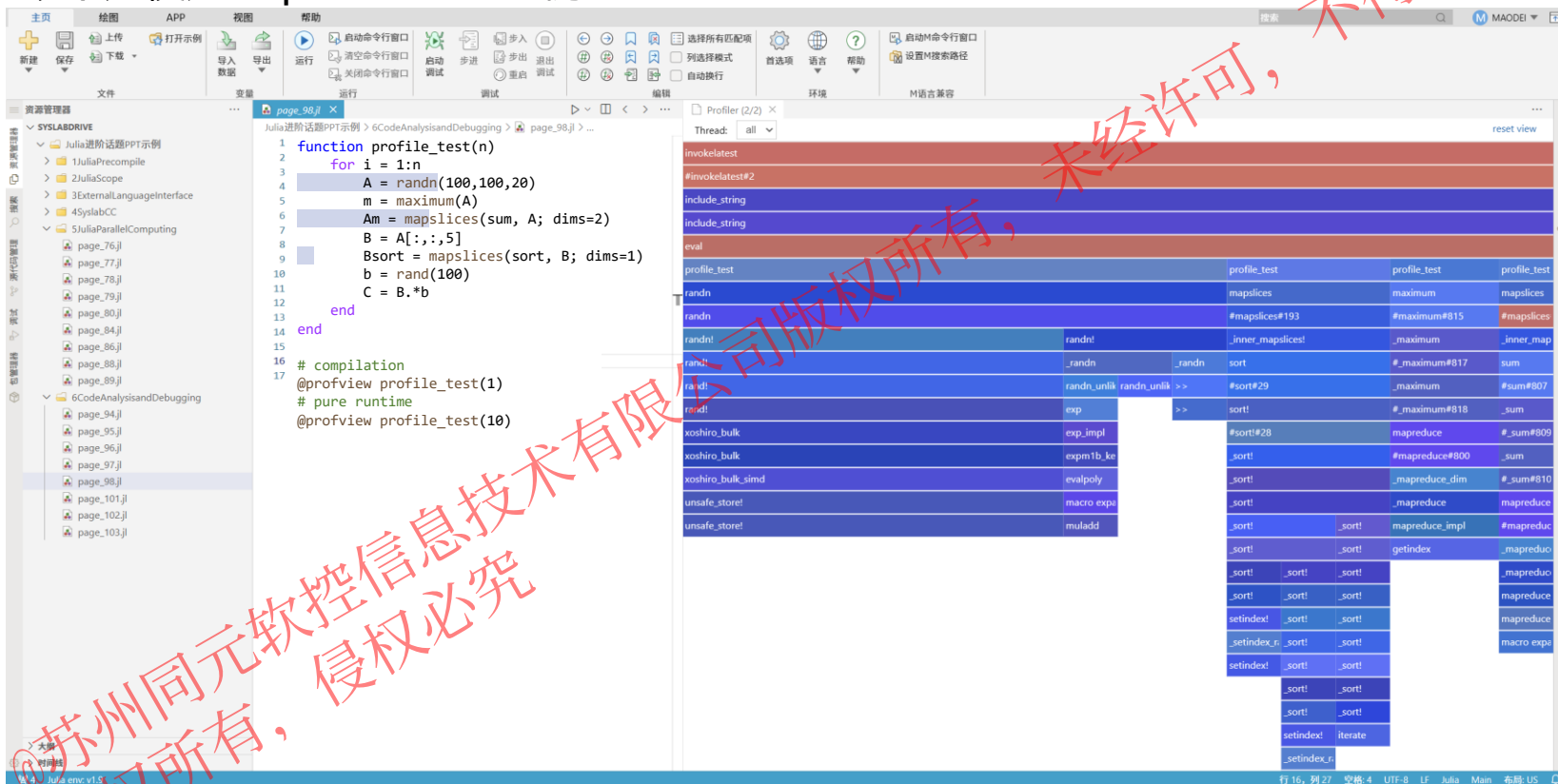
@profview

Syslab 内置 Profiler 模块，用于性能分析。同时以火焰图的形式，对性能采样分析结果进行可视化展示。

```
@profview expression
```

- expression 是要性能分析的代码块。

以下是使用 @profview 的示例：



Profiler 模块是基于采样实现，因此对于小函数（ms 以内执行完），需要多次执行才能得到一个相对可靠的分析结果。

02 代码动态更新

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

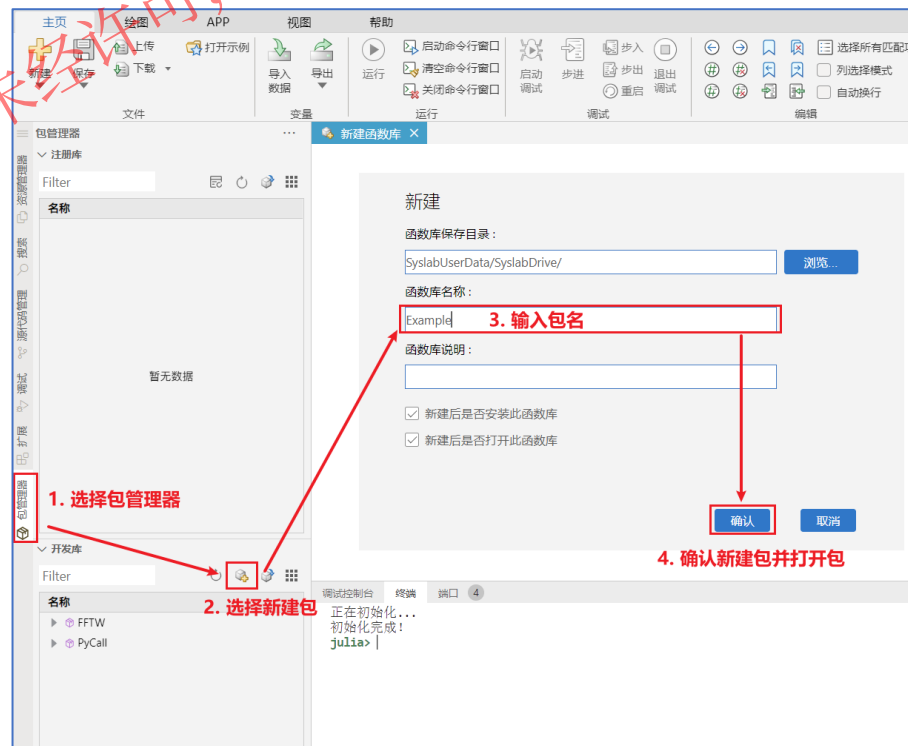
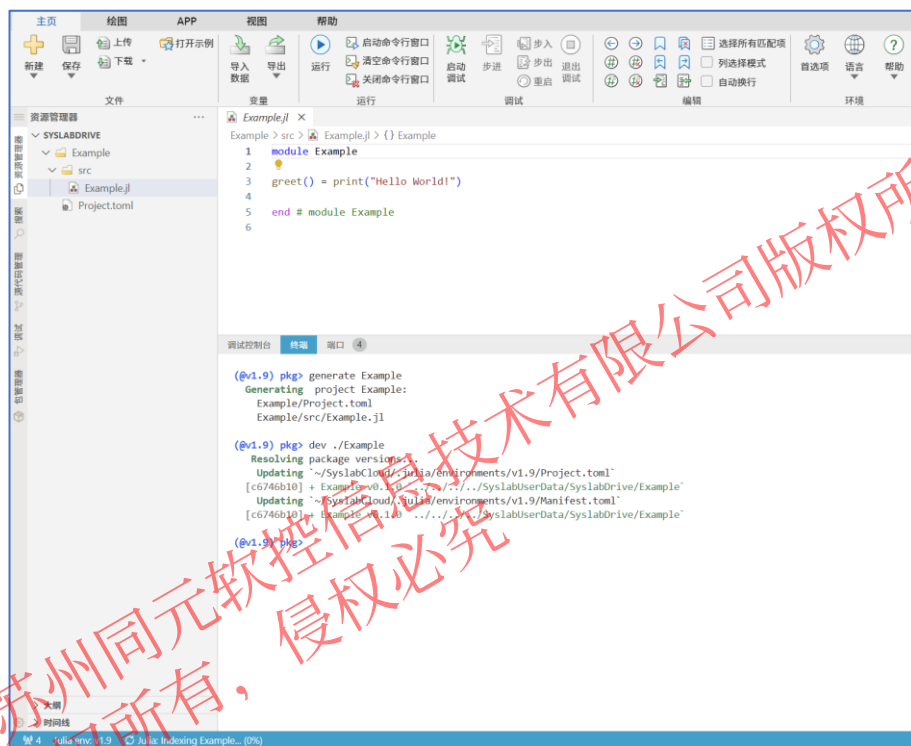
2. 代码动态更新

Revise

- Revise.jl 允许您修改代码并使用更改，而无需重新启动 Julia。
- MWORKS.Syslab 默认安装了 Revise 库。

新建 Julia 包：

- 使用在包管理器模式下使用 generate 命令新建包，使用 dev 命令 安装包
- 使用 Syslab 的 包管理器 面板新建并安装包



2. 代码动态更新

Revise

以下为 Revise 的使用示例：

- 例1：常规使用

1. 在 Julia REPL 加载 Revise 和 新建的包，**确保在加载其他包之前加载 Revise**

```
using Revise
using Example
```

2. Example.jl 中还不存在一个名为 f 的函数：

```
Example.f()
```

3. 在 Example.jl 中添加一个函数 $f() = \pi$ 并保存文件，即可在 Julia REPL 中使用该函数：

```
julia> using Revise
julia> using Example
```

```
julia> Example.f()
ERROR: UndefVarError: `f` not defined
Stacktrace:
 [1] getproperty(x::Module, f::Symbol)
    @ Base ./Base.jl:31
 [2] top-level scope
    @ REPL[5]:1
```

```
Example.jl ×
Example > src > Example.jl > {} Example > f
1 module Example
2
3 greet() = print("Hello World!")
4 f() = pi
5 end # module Example
6
```

```
调试控制台 终端 端口 4

julia> Example.f()
π = 3.1415926535897...
```

2. 代码动态更新

Revise

以下为 Revise 的使用示例：

- 例2：修改结构体

1. 首先，在 Example.jl 示例基础上，增加一段代码：

```
export FooStruct, processFoo

# 版本1
struct FooStruct1
    bar::Int
end
FooStruct = FooStruct1

function processFoo(foo::FooStruct)
    @info foo.bar
end
```

2. 其次，若想修改结构体，如新增一个成员。此时，不需要重启REPL，只需执行 `revise(Module)`，即可生效：

```
# 版本2
struct FooStruct2 # change version here
    bar::Int
    str::String # new add
end
FooStruct = FooStruct2
```

```
julia> using Revise
julia> using Example
julia> processFoo(FooStruct(1))
[ Info: 1
```

```
julia> revise(Example)
true

julia> processFoo(FooStruct(1, "x"))
[ Info: 1
```

2. 代码动态更新

Revise

以下为 Revise 的使用示例：

- 例2：修改结构体
- 3. 最后，当我们结构体设计稳定后，就可以使用正式名称，并重启REPL。

```
# 结构体稳定后，改为正式名称
struct FooStruct
  bar::Int
  str::String
end
# FooStruct = FooStruct2 # 注释临时代码
```

Revise 的局限性：

- 类型定义或常量的更改
- 变量和函数同名引起的冲突
- 导出项的移除

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听